Numeric Data

Unit 3

Supervised Learning

Supervised Learning is a type of machine learning where algorithms are trained on labeled data to make predictions or classify new data.

This approach involves using a dataset that includes both input features and their corresponding output labels.

The model learns from these labeled examples to identify patterns and relationships between inputs and outputs, enabling it to predict outputs for unseen data.

Classification problems

Predicting the category or class of an observation or data point is the goal of a classification issue in machine learning. The objective is to create a model that can learn to categorize fresh data based on correlations and patterns found in the training data.

The model is trained with labeled data, where the proper class or category of each observation is known, since classification is a supervised learning technique. The class of fresh, unknown data is then predicted using the model.

Classification issues might be binary—where there are only two classes that could exist—or multiclass—where there are multiple classes that could exist. Email spam categorization, image recognition, sentiment analysis, and disease diagnosis are a few typical instances of classification issues.

Classification Problem

There are many algorithms that can be used for classification, including logistic regression, decision trees, support vector machines (SVMs), random forests, and neural networks. The choice of algorithm depends on factors such as the size and complexity of the data, the number of classes, and the accuracy and interpretability requirements of the model.

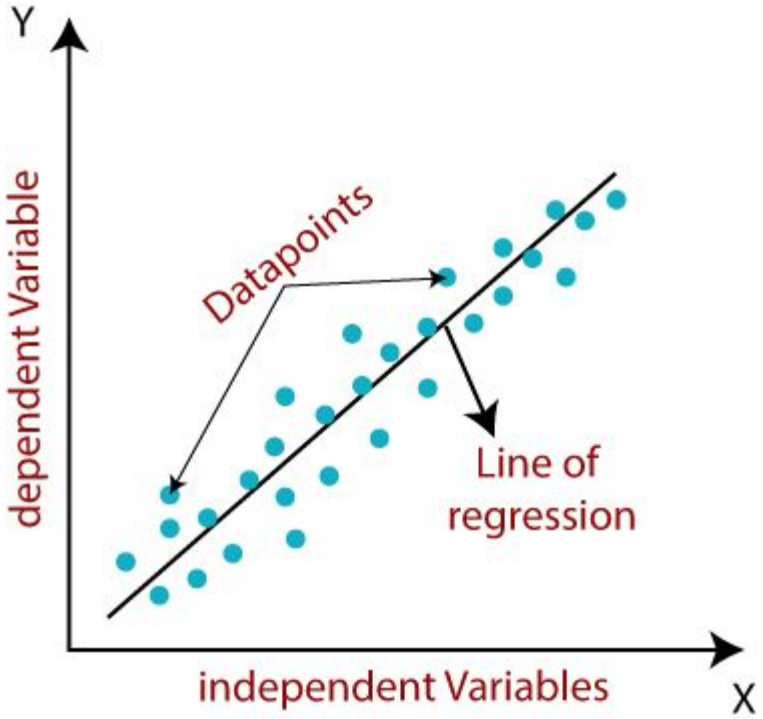
Linear Regression in Machine Learning

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price,** etc.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (x) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

Linear Regression (cont.)

The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



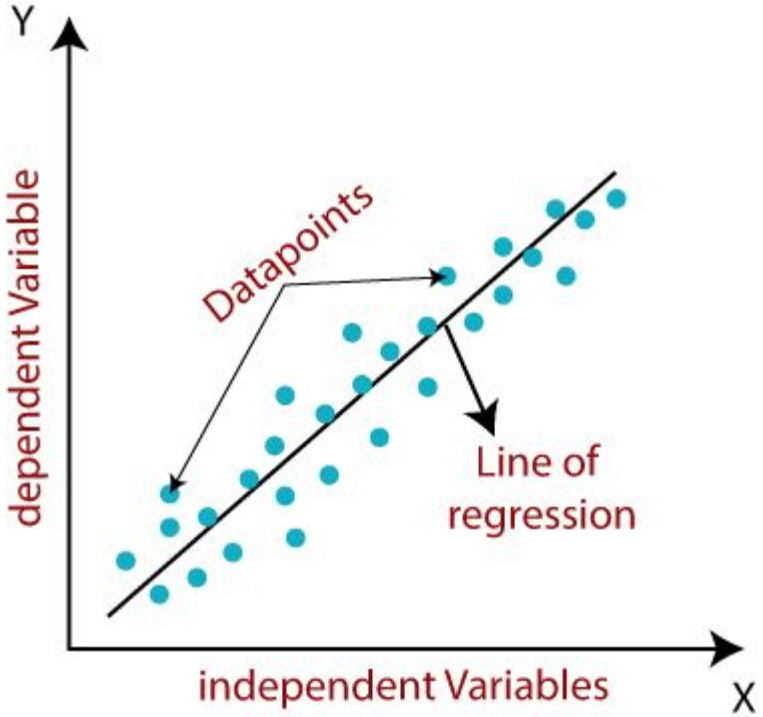
Linear Regression (cont.)

Mathematically, we can represent linear regression as:

$$y= a_0+a_1x+ \varepsilon$$

Y= Dependent Variable (Target Variable)
X= Independent Variable (predictor Variable)
a0= intercept of the line (Gives an additional degree of freedom)
a1 = Linear regression coefficient (scale factor to each input value).
ε = random error

The values for x and y variables are training datasets for Linear Regression model representation.



Types of Linear Regression

Linear regression can be further divided into two types of the algorithm:

- **Simple Linear Regression:**

  If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
- **Multiple Linear regression:**

  If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

Finding the best fit-line

When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

The different values for weights or the coefficient of lines (a_0, a_1) gives a different line of regression, so we need to calculate the best values for a_0 and a_1 to find the best fit line, so to calculate this we use cost function.

Cost function-

- The different values for weights or coefficient of lines (a_0, a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

$$MSE = 1\frac{1}{N}\sum_{i=1}^{n}(y_i - (a_1x_i + a_0))^2$$

Where,

N=Total number of observation
Yi = Actual value
$(a1x_i+a_0)$= Predicted value.

Problem Deninition:

Find a quadratic regression model for the following data:

X	Y
1	1
2	2
3	1.3
4	3.75
5	2.25

Solution:

Let the simple linear regression model be

$$y = a + bx$$

Steps to find **a** and **b,**

First, find the mean and covariance.

Means of x and y are given by,

$$\bar{x} = \frac{1}{n}\sum x_i$$

$$\bar{y} = \frac{1}{n}\sum y_i$$

The variance of x is given by,

$$\mathrm{Var}\,(x) = \frac{1}{n-1}\sum(x_i - \bar{x}_i)^2$$

The covariance of x and y, denoted by Cov(x, y)is defined as,

$$\mathrm{Cov}(x, y) = \frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})$$

Now the values of a and b can be computed using the following formulas:

$$b = \frac{\mathrm{Cov}\,(x, y)}{\mathrm{Var}\,(x)}$$

$$a = \bar{y} - b\bar{x}$$

First, find the mean of x and y,

$$
\begin{aligned}
n &= 5 \\
\bar{x} &= \frac{1}{5}(1.0 + 2.0 + 3.0 + 4.0 + 5.0) \\
&= 3.0 \\
\bar{y} &= \frac{1}{5}(1.00 + 2.00 + 1.30 + 3.75 + 2.25) \\
&= 2.06
\end{aligned}
$$

Next, find the Covariance between x and y,

$$
\operatorname{Cov}(x, y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y})
$$

$$
\begin{aligned}
\operatorname{Cov}(x, y) &= \frac{1}{4}[(1.0 - 3.0)(1.00 - 2.06) + \cdots + (5.0 - 3.0)(2.25 - 2.06)] \\
&= 1.0625
\end{aligned}
$$

Now find the variance of x,

$$\text{Var}(x) = \frac{1}{n-1}\sum(x_i - \bar{x}_i)^2$$

$$\text{Var}(x) = \frac{1}{4}[(1.0-3.0)^2 + \cdots + (5.0-3.0)^2]$$
$$= 2.5$$

Now, find the intercept and coefficients,

$$b = \frac{\text{Cov}(x,y)}{\text{Var}(x)}$$

$$a = \bar{y} - b\bar{x}$$

$$b = \frac{1.0625}{2.5}$$
$$= 0.425$$

$$a = 2.06 - 0.425 \times 3.0$$
$$= 0.785$$

Therefore, the linear regression model for the data is,

$$y = 0.785 + 0.425\,x$$

Multiple Linear Regression

Multiple linear regression is used to estimate the relationship between two or more independent variables and one dependent variable. You can use multiple linear regression when you want to know:

1. How strong the relationship is between two or more independent variables and one dependent variable (e.g. how rainfall, temperature, and amount of fertilizer added affect crop growth).
2. The value of the dependent variable at a certain value of the independent variables (e.g. the expected yield of a crop at certain levels of rainfall, temperature, and fertilizer addition).

Multiple linear regression formula

The formula for a multiple linear regression is:

$$y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n + \epsilon$$

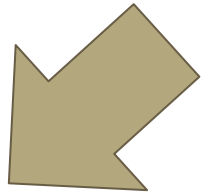
- y = the predicted value of the dependent variable
- B_0 = the y-intercept (value of y when all other parameters are set to 0)
- $B_1 X_1$ = the regression coefficient (B_1) of the first independent variable (X_1) (a.k.a. the effect that increasing the value of the independent variable has on the predicted y value)
- ... = do the same for however many independent variables you are testing
- $B_n X_n$ = the regression coefficient of the last independent variable
- ϵ = model error (a.k.a. how much variation there is in our estimate of y)

- The multiple regression of two variables x1 and x2 is given as follows:

$$\boldsymbol{y = f(x_1, x_2)}$$

$$\boldsymbol{y = a_0 + a_1 x_1 + a_2 x_2}$$

- In general, this is given for 'n' independent variables as:



$$\boldsymbol{y = f(x_1, x_2, \ldots . x_n)}$$

$$\boldsymbol{y = a_0 + a_1 x_1 + a_2 x_2 + \ldots \ldots + a_n x_n + \varepsilon}$$

- Here, $x_1, x_2, \ldots . x_n$ are predictor variables, y is the dependent variable, $(a_0, a_1, a_2, \ldots . a_n)$ are the coefficients of the regression equation and ε is the error term.

Data having multiple independent variables

X1 (Product 1 Sales)	X2 (Product 2 Sales)	Y (Weekly Sales)
1	4	1
2	5	6
3	8	8
4	2	12

Representing the above data in matrix form

- Here, the matrices for Y and X are given as follows:

$$X = \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} \text{ and } Y = \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix}$$

The coefficient of multiple linear regression equation is given as

$$a = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix}.$$

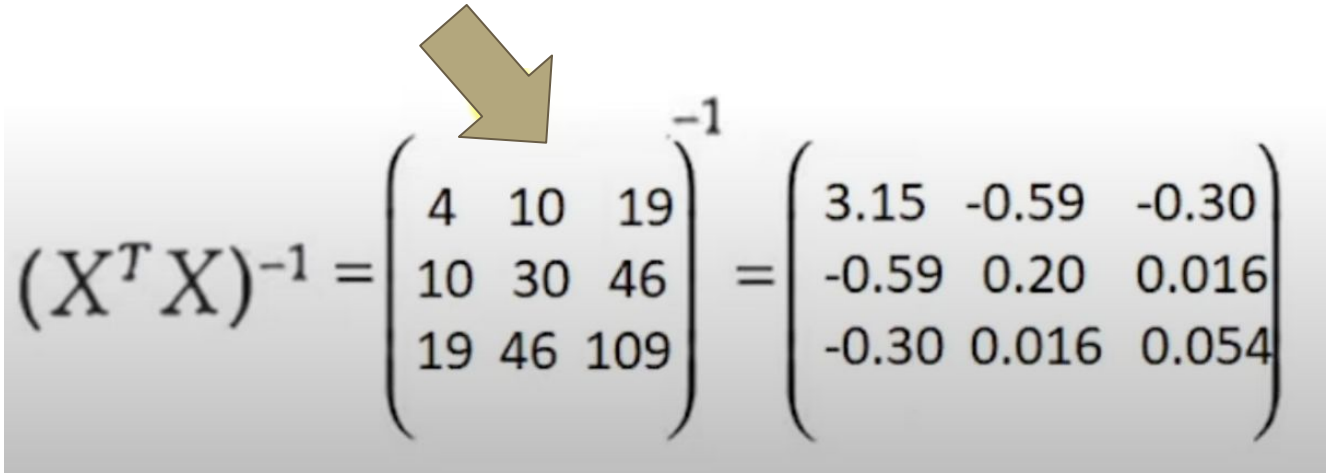
- The regression coefficient for multiple regression is calculated the same way as linear regression:

$$\hat{a} = ((X^T X)^{-1} X^T) Y$$

Calculation of first part

$$X^T X = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 10 & 19 \\ 10 & 30 & 46 \\ 19 & 46 & 109 \end{pmatrix}$$

Calculation of the inverse



Evaluating the second part

$$X^T X)^{-1} X^T = \begin{pmatrix} 3.15 & -0.59 & -0.30 \\ -0.59 & 0.20 & 0.016 \\ -0.30 & 0.016 & 0.054 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} = \begin{pmatrix} 0.05 & 0.47 & -1.02 & 0.19 \\ -0.32 & -0.098 & 0.155 & 0.26 \\ -0.065 & 0.005 & 0.185 & -0.125 \end{pmatrix}$$

Evaluation of regression coefficient

$$\hat{a} = ((X^T X)^{-1} X^T) Y = \begin{pmatrix} 0.05 & 0.47 & -1.02 & 0.19 \\ -0.32 & -0.098 & 0.155 & 0.26 \\ -0.065 & 0.005 & 0.185 & -0.125 \end{pmatrix} \times \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix} = \begin{pmatrix} -1.69 \\ 3.48 \\ -0.05 \end{pmatrix}$$

$$a_0 = -1.69$$
$$a_1 = 3.48$$
$$a_2 = -0.05$$

Final Equation of Multiple Linear Regression

- $y = a_0 + a_1 x_1 + a_2 x_2$

- Hence, the constructed model is:

- $y = -1.69 + 3.48 x_1 - 0.05 x_2$

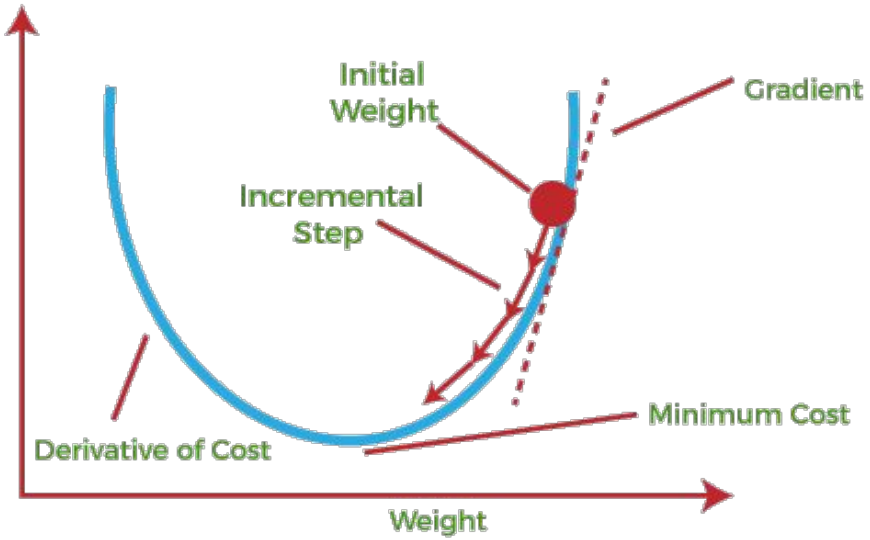
Gradient Descent

Gradient Descent is defined as one of the most commonly used iterative optimization algorithms of machine learning to train the machine learning and deep learning models.

It helps in finding the local minimum of a function.

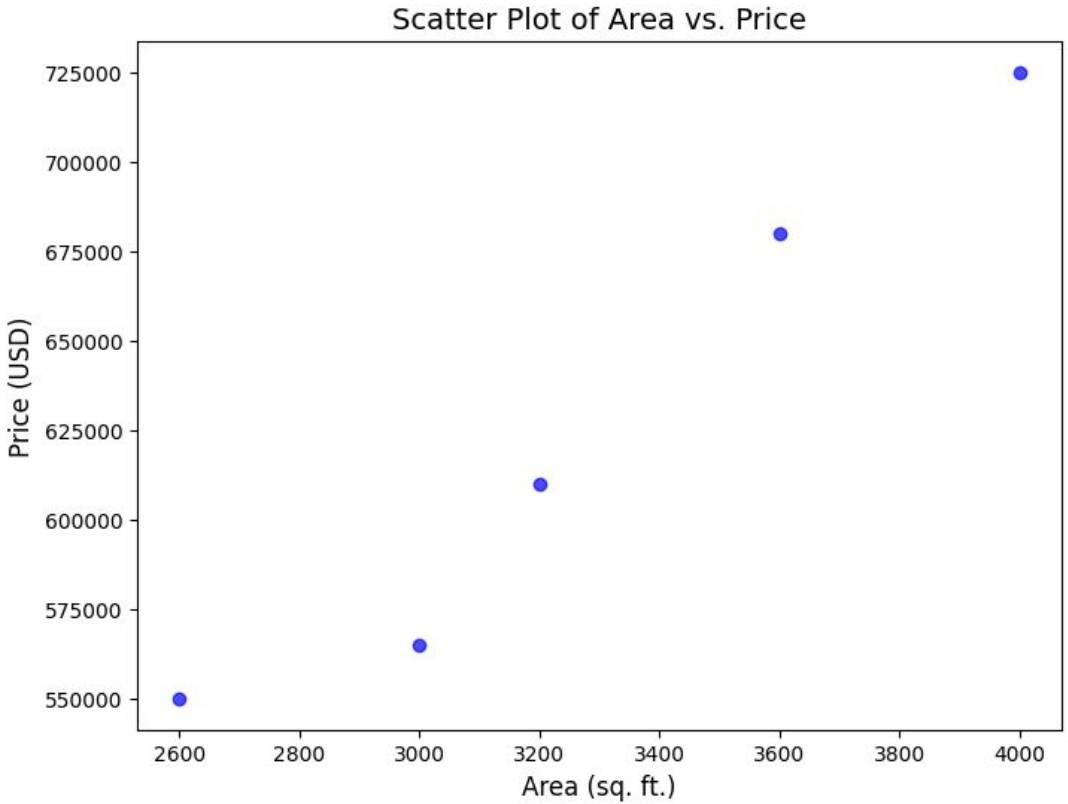
Gradient Descent

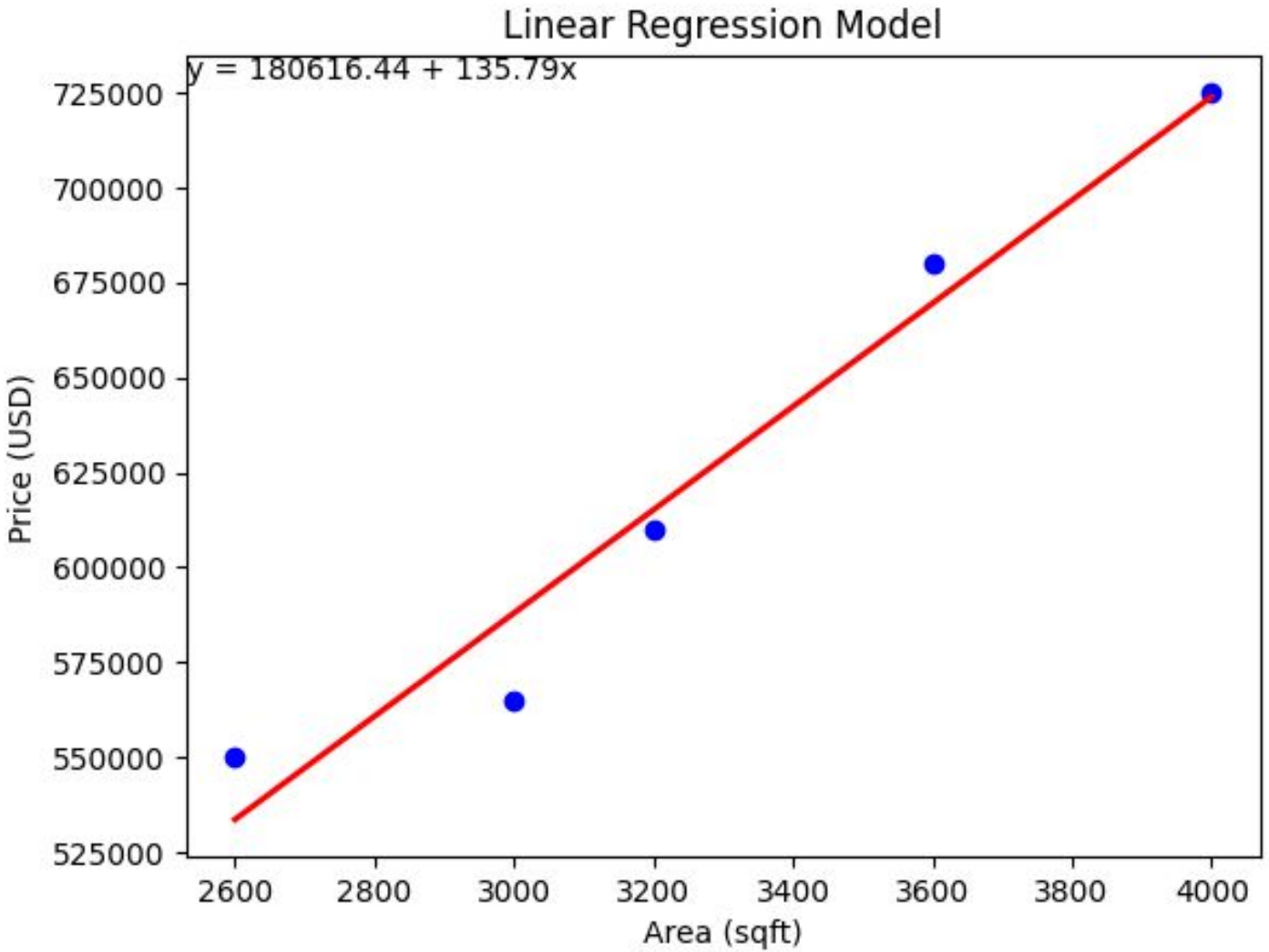
- If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the **local minimum** of that function.
- Whenever we move towards a positive gradient or towards the gradient of the function at the current point, we will get the **local maximum** of that function.

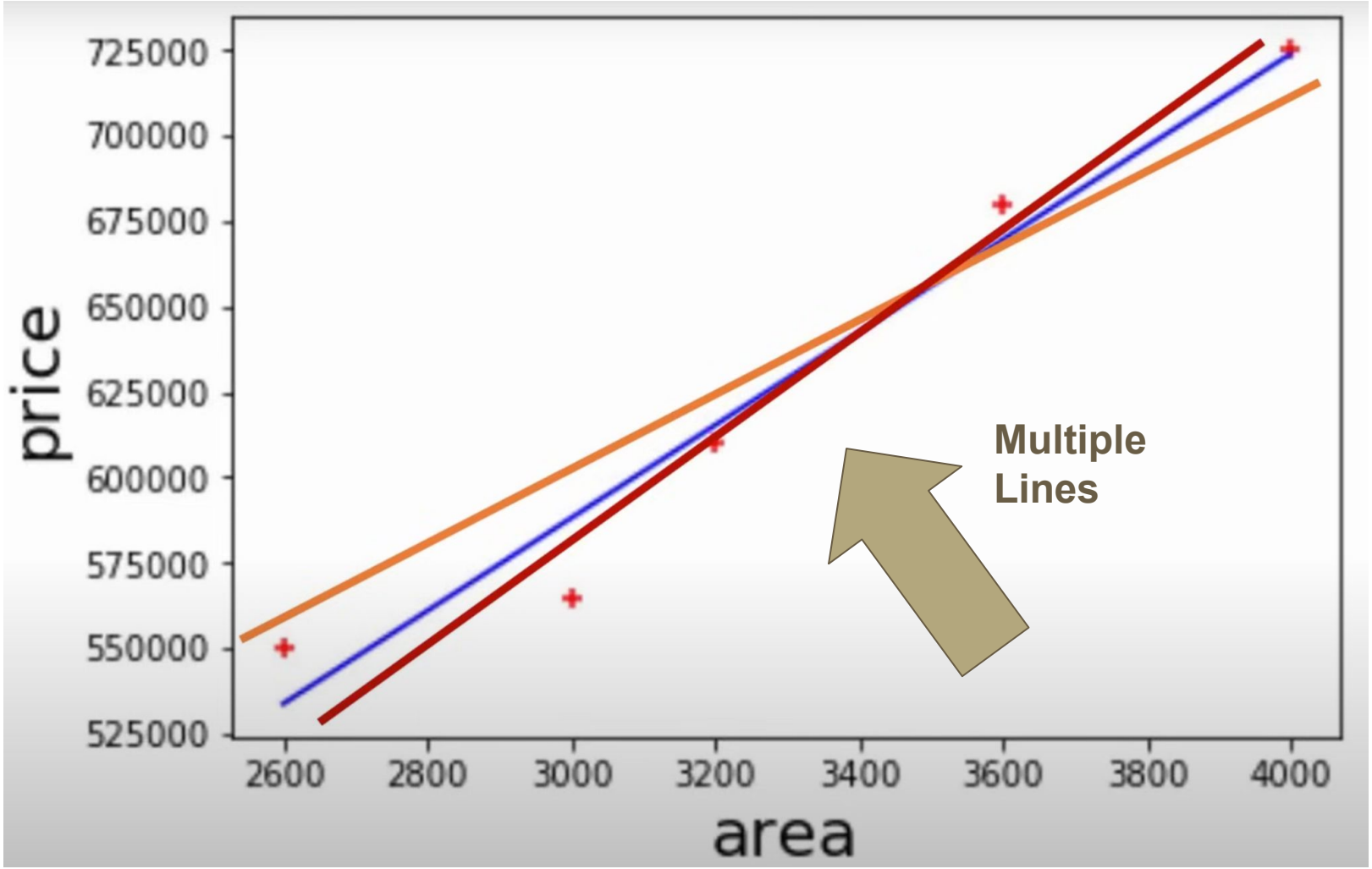


For a given Data set

area	price
2600	550000
3000	565000
3200	610000
3600	680000
4000	725000




Linear Regression Model
y = 180616.44 + 135.79x
Price (USD)
725000
700000
675000
650000
625000
600000
575000
550000
525000
2600
2800
3000
3200
3400
3600
3800
4000
Area (sqft)


725000
700000
675000
650000
625000
600000
575000
550000
525000
price
2600
2800
3000
3200
3400
3600
3800
4000
area
Multiple
Lines

Mean Squared Error

$$mse = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - y_{predicted}\right)^2$$

$$mse = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - (mx_i + b)\right)^2$$

Cost Function

Gradient Descent is an algorithm that finds the best fit line for the given training data set

The main objective of using a gradient descent algorithm is to minimize the cost function using iteration. To achieve this goal, it performs two steps iteratively:

- Calculates the first-order derivative of the function to compute the gradient or slope of that function.
- Move away from the direction of the gradient, which means slope increased from the current point by alpha times, where Alpha is defined as Learning Rate. It is a tuning parameter in the optimization process which helps to decide the length of the steps.

Cost Function

The cost function is defined as the measurement of difference or error between actual values and expected values at the current position and present in the form of a single real number.

It helps to increase and improve machine learning efficiency by providing feedback to this model so that it can minimize error and find the local or global minimum.

Further, it continuously iterates along the direction of the negative gradient until the cost function approaches zero. At this steepest descent point, the model will stop learning further.

Cost Function

At this steepest descent point, the model will stop learning further.

Although cost function and loss function are considered synonymous, also there is a minor difference between them.

The slight difference between the loss function and the cost function is about the error within the training of machine learning models, as loss function refers to the error of one training example, while a cost function calculates the average error across an entire training set.

The cost function is calculated after making a hypothesis with initial parameters and modifying these parameters using gradient descent algorithms over known data to reduce the cost function.

How does gradient descent works?

Before starting the working principle of gradient descent, we should know some basic concepts to find out the slope of a line from linear regression.

The equation for simple linear regression is given as:

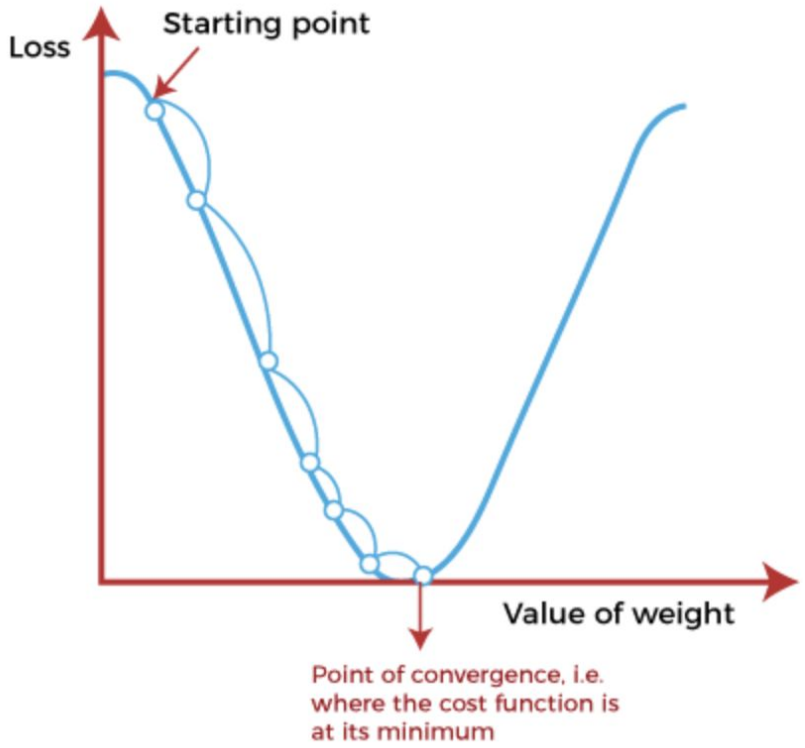
$Y=mX+c$

Where 'm' represents the slope of the line, and 'c' represents the intercepts on the y-axis.

The starting point(shown in fig.) is used to evaluate the performance as it is considered just as an arbitrary point. At this starting point, we will derive the first derivative or slope and then use a tangent line to calculate the steepness of this slope. Further, this slope will inform the updates to the parameters (weights and bias).

The slope becomes steeper at the starting point or arbitrary point, but whenever new parameters are generated, then steepness gradually reduces, and at the lowest point, it approaches the lowest point, which is called **a point of convergence.**

The main objective of gradient descent is to minimize the cost function or the error between expected and actual. To minimize the cost function, two data points are required:



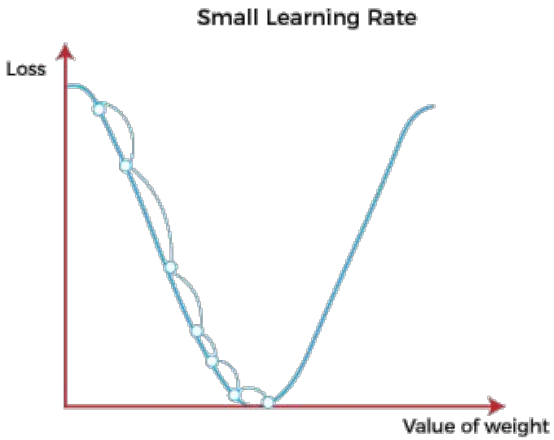
Learning Rate

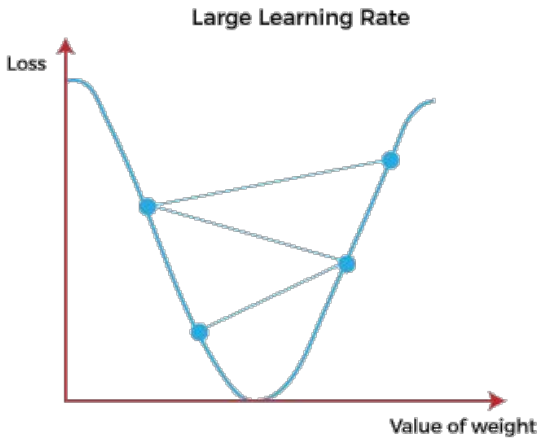
It is defined as the step size taken to reach the minimum or lowest point.

This is typically a small value that is evaluated and updated based on the behavior of the cost function.

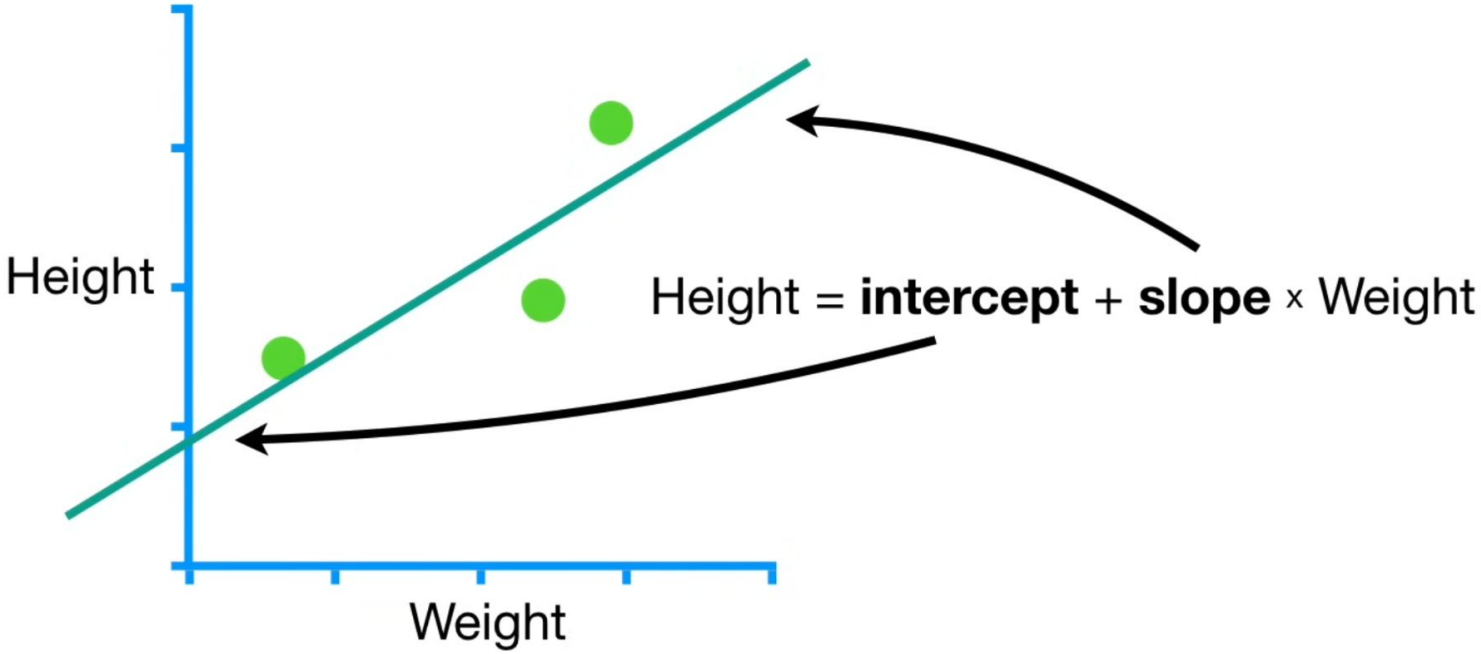
If the learning rate is high, it results in larger steps but also leads to risks of overshooting the minimum.

At the same time, a low learning rate shows the small step sizes, which compromises overall efficiency but gives the advantage of more precision.

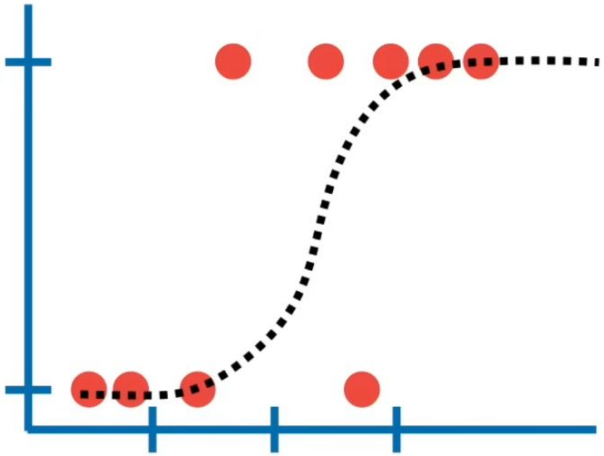


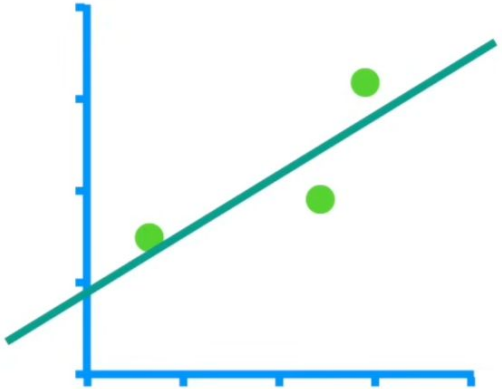


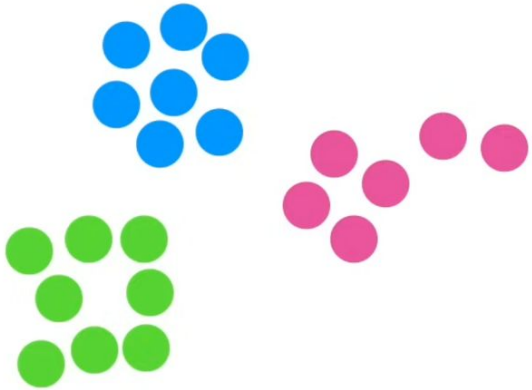
When we fit a line with **Linear Regression**, we optimize the **Intercept** and **Slope**.



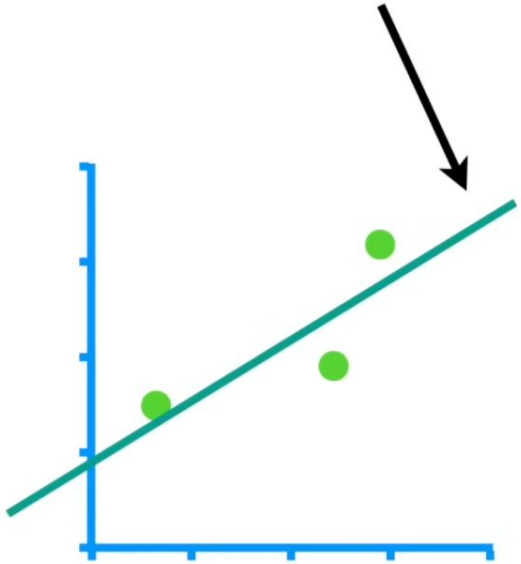
These are just a few examples of the stuff we optimize, there are tons more.

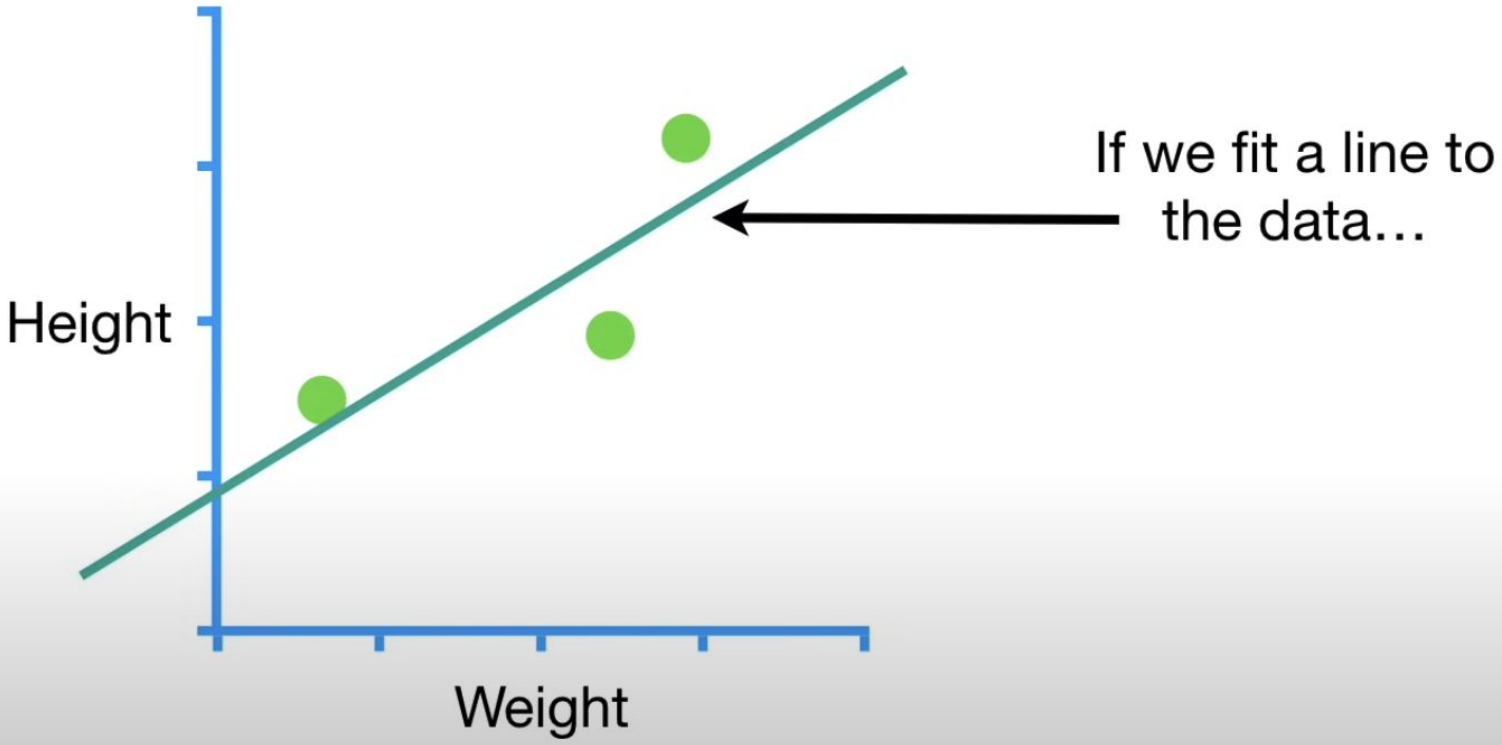




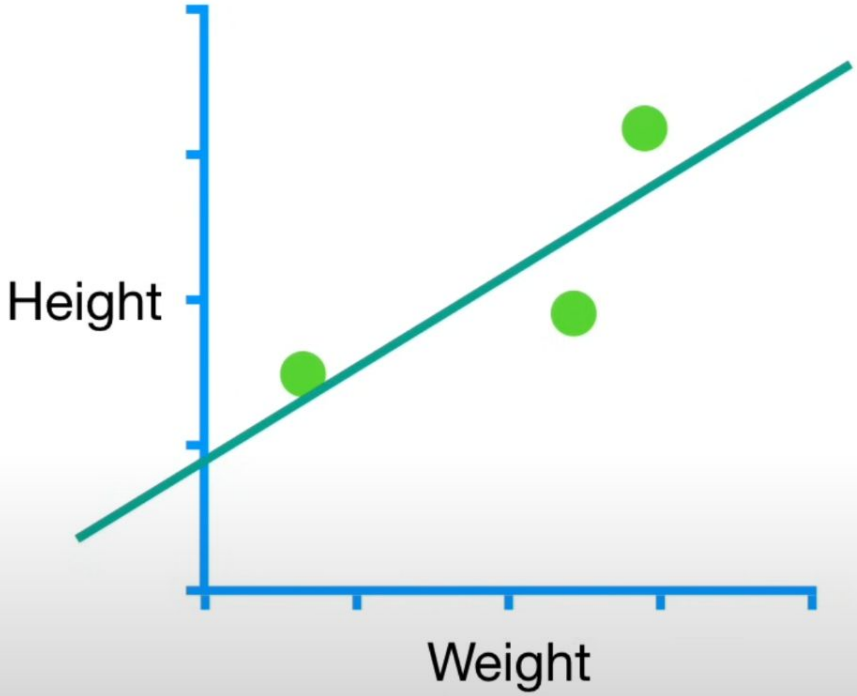


So if we learn how to optimize this line using **Gradient Descent**…

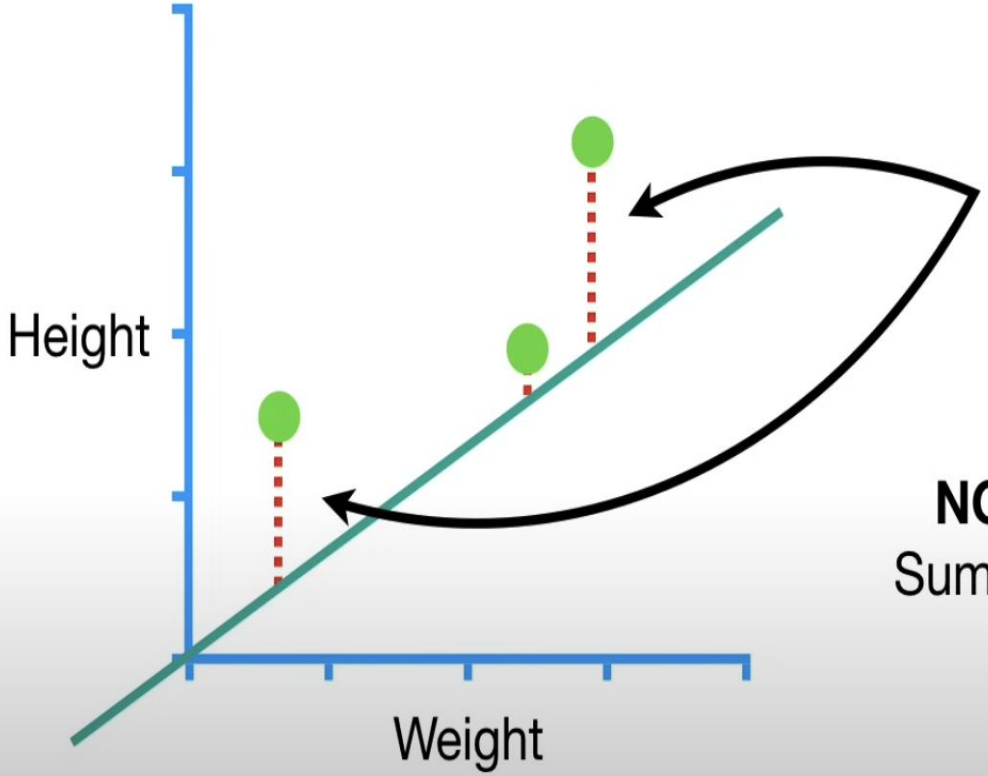



Height
Weight
If we fit a line to
the data…

Predicted Height = intercept + slope × **Weight**

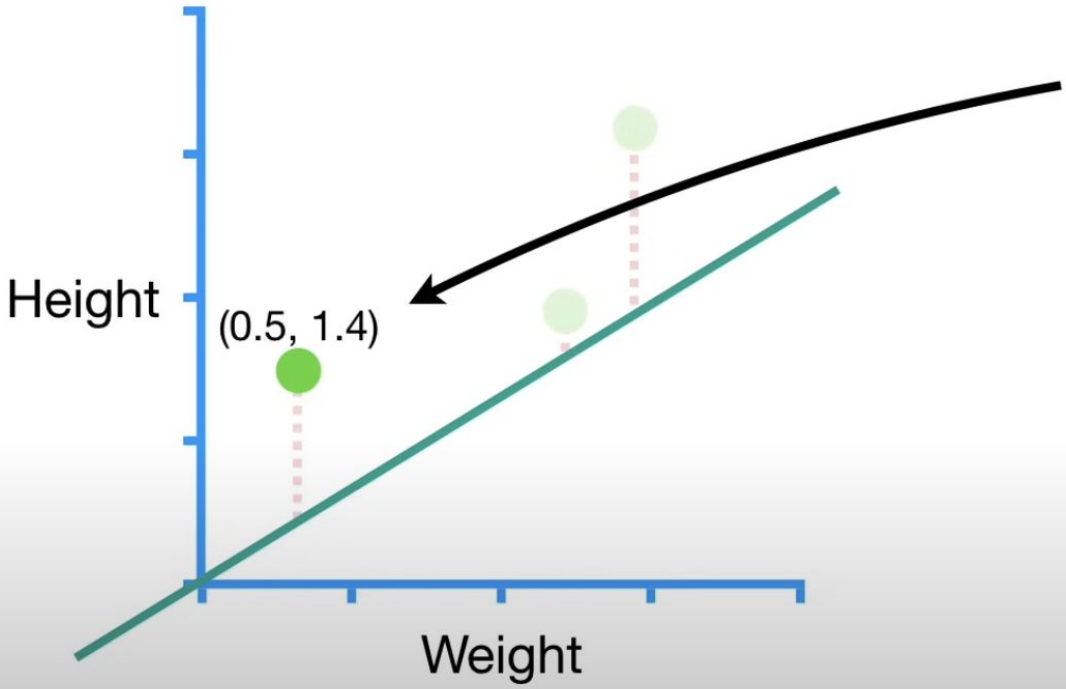


So let's learn how **Gradient Descent** can fit a line to data by finding the optimal values for the **Intercept** and the **Slope**.

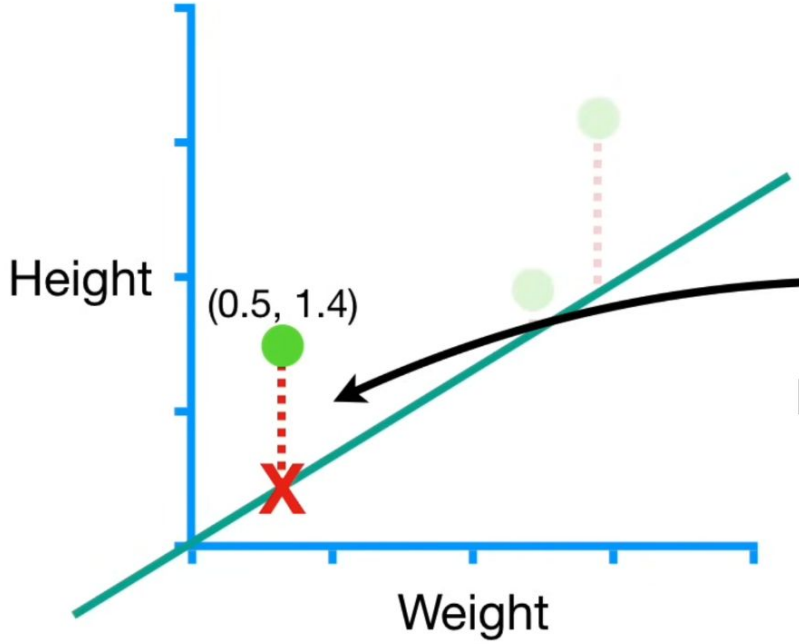


In this example, we will evaluate how well this line fits the data with the **Sum of the Squared Residuals**.

NOTE: In Machine Learning lingo, The Sum of the Squared Residuals is a type of **Loss Function**.



This datapoint represents a person with **Weight 0.5** and **Height 1.4**.

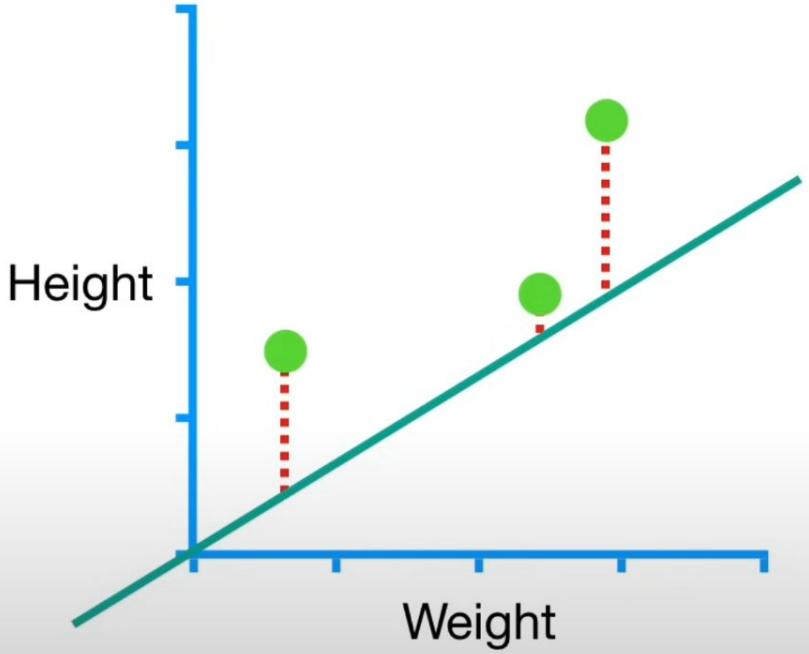


…and that gives us **1.1** for the residual.

Residual = 1.4 - 0.32 = **1.1**

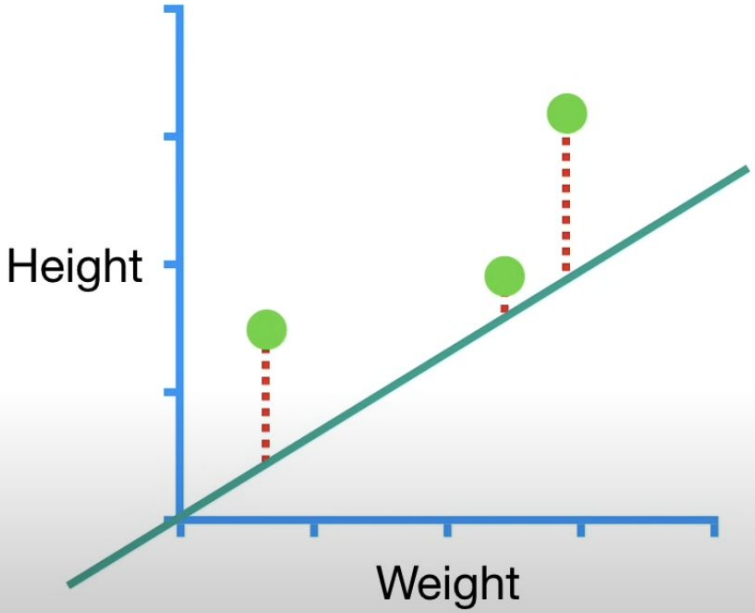
Predicted Height = 0 + 0.64 × **0.5** = **0.32**

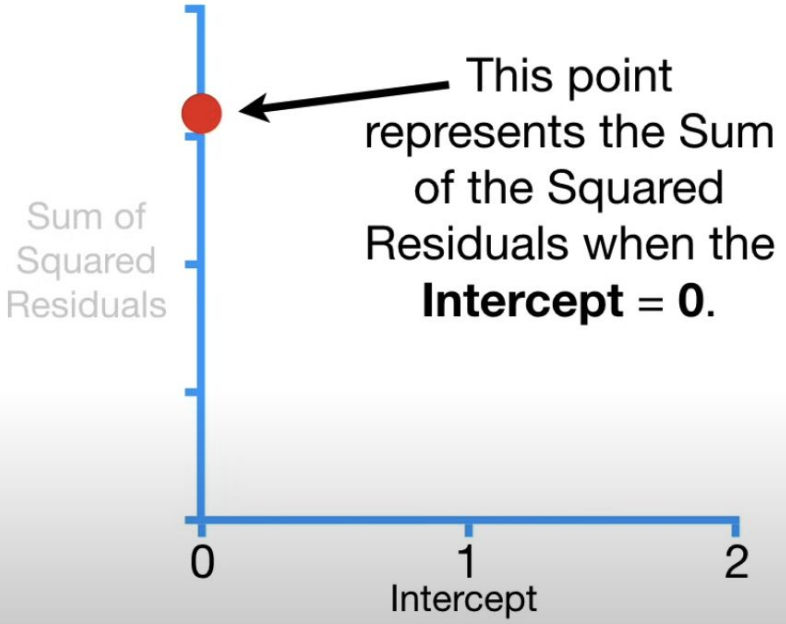
Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2 = 3.1$



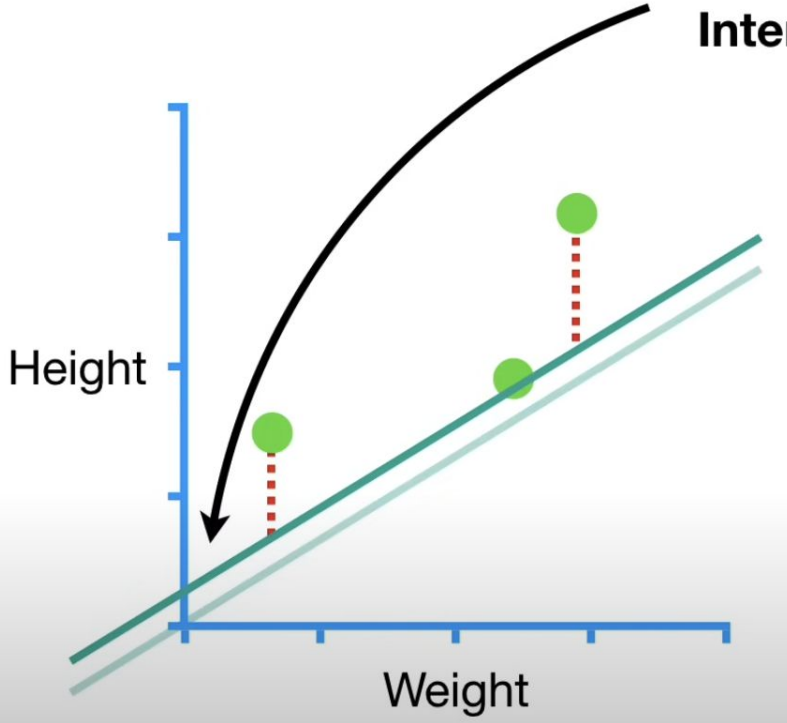
In the end, **3.1** is the Sum of the Squared Residuals.

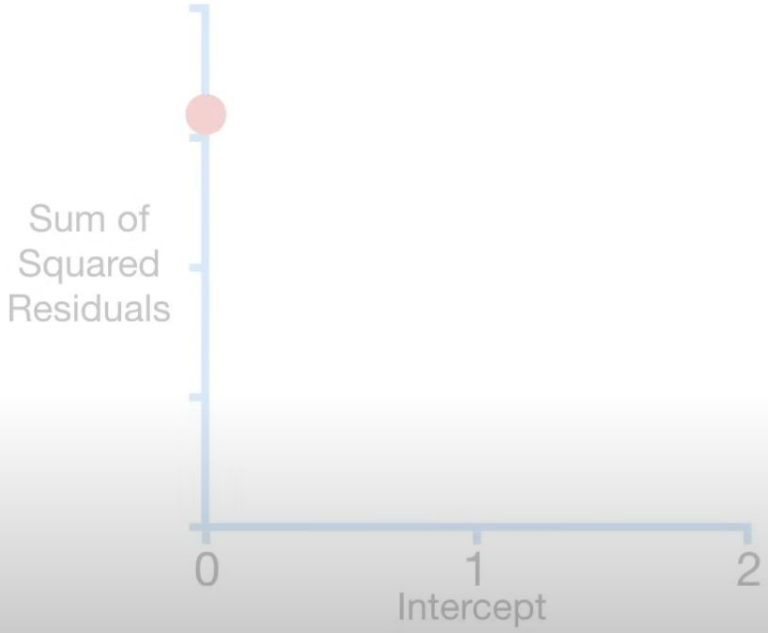
Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2 = 3.1$

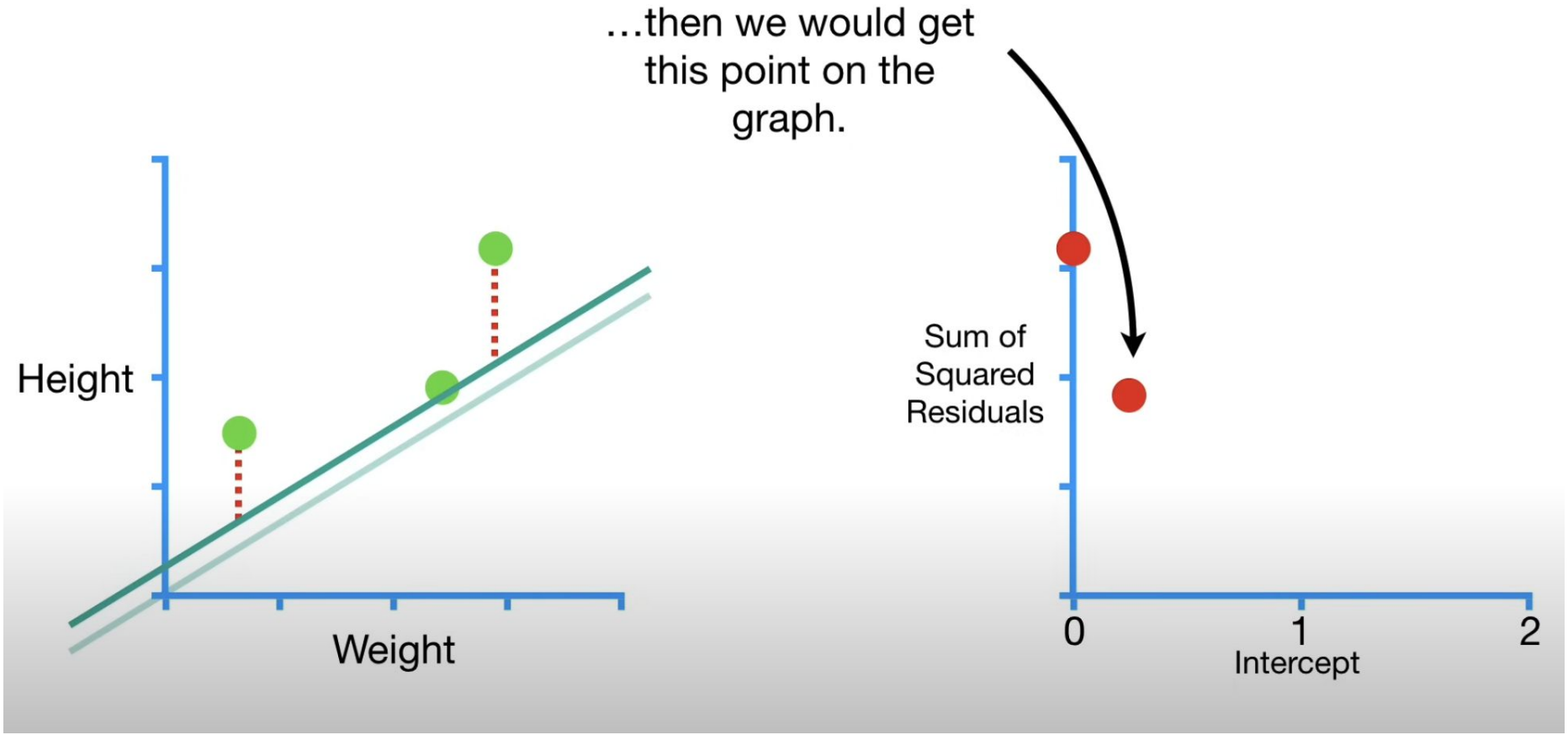




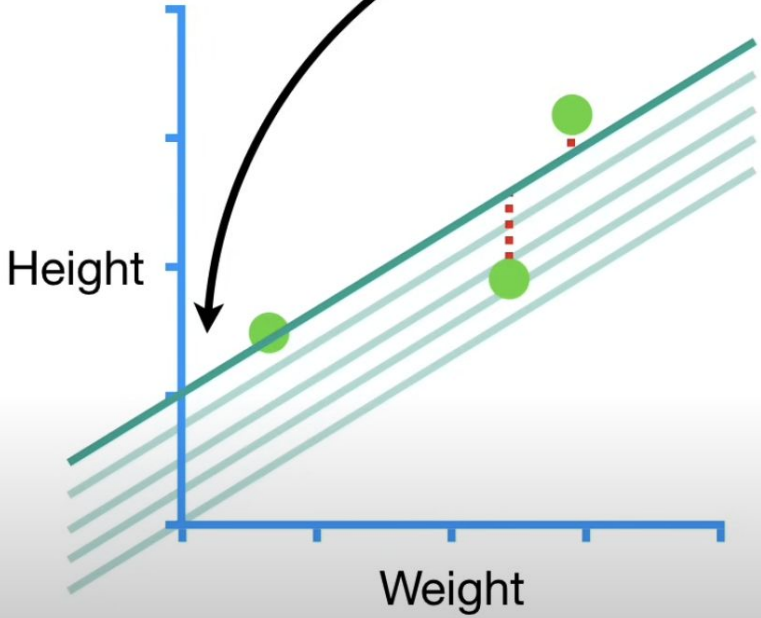
However, if the
Intercept = 0.25…

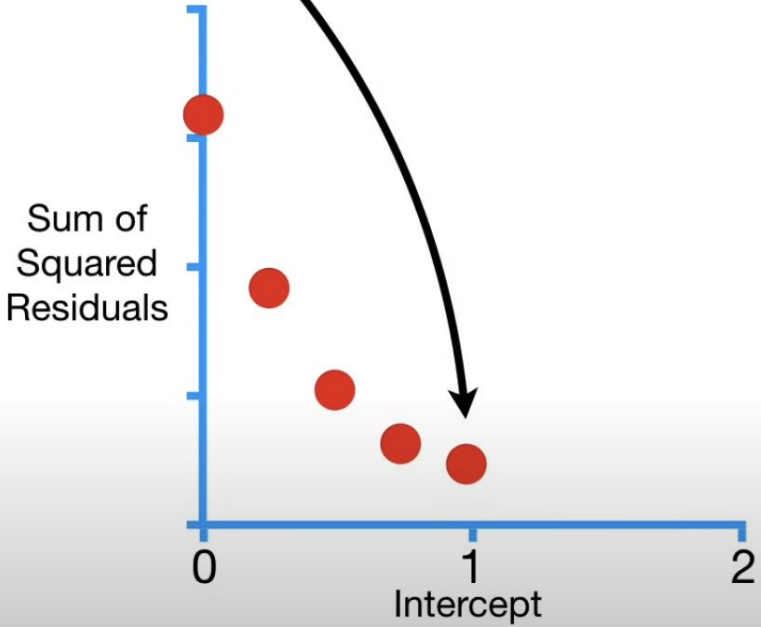


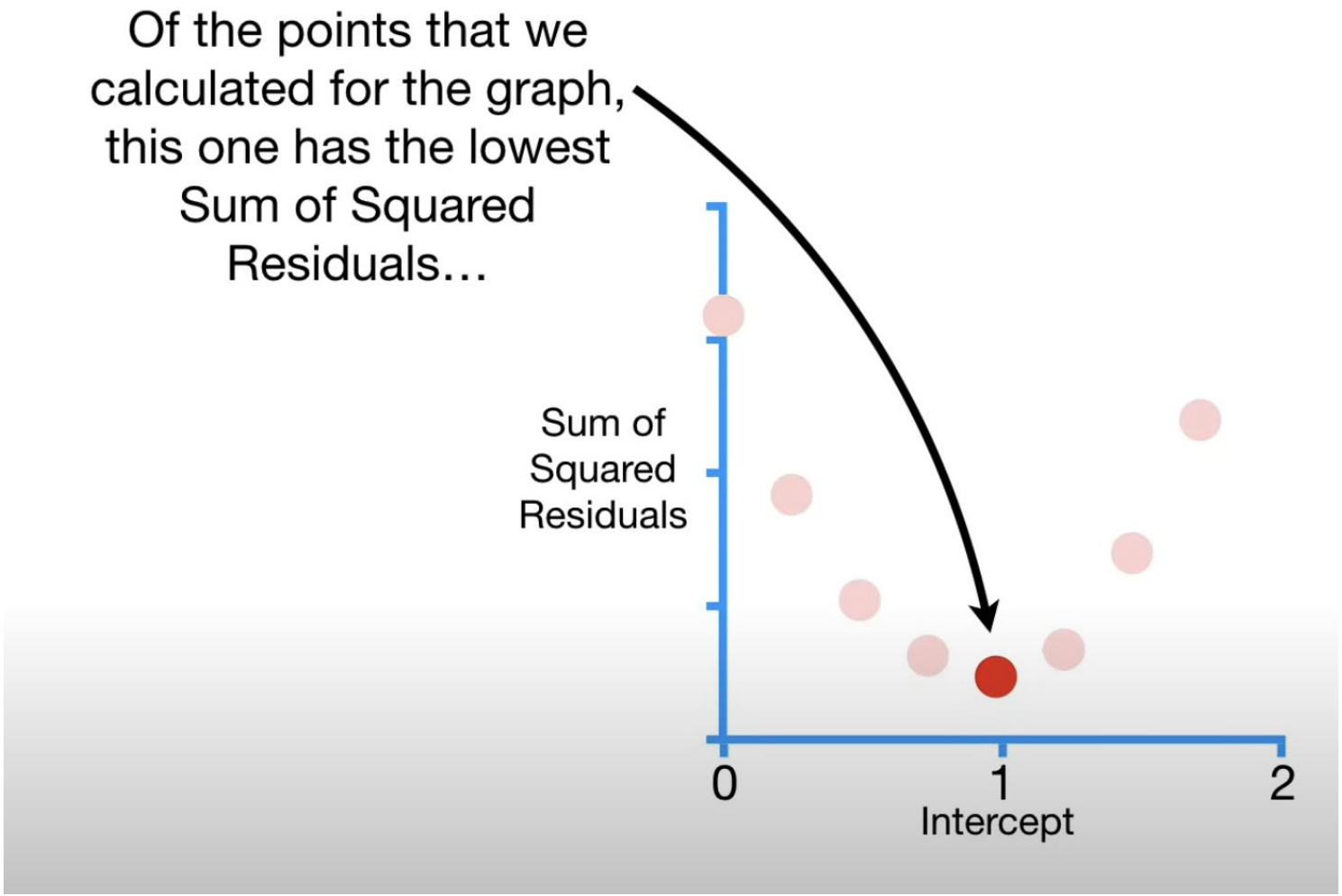



…then we would get this point on the graph.
Height
Weight
Sum of Squared Residuals
0
1
2
Intercept

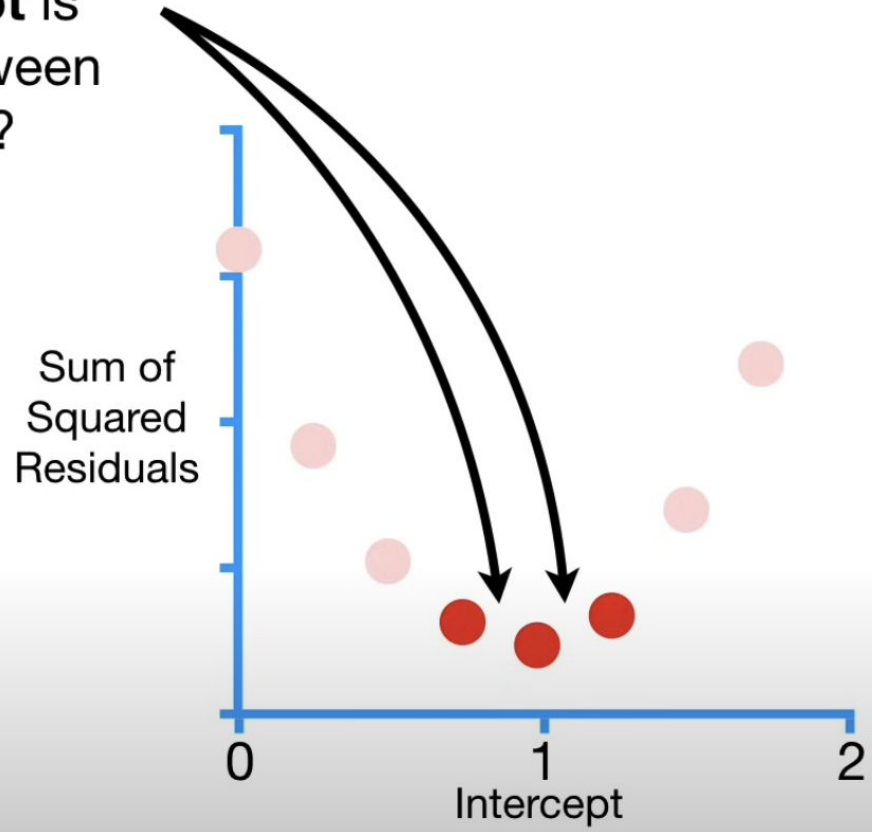
And for increasing values for the **Intercept**, we get these points.



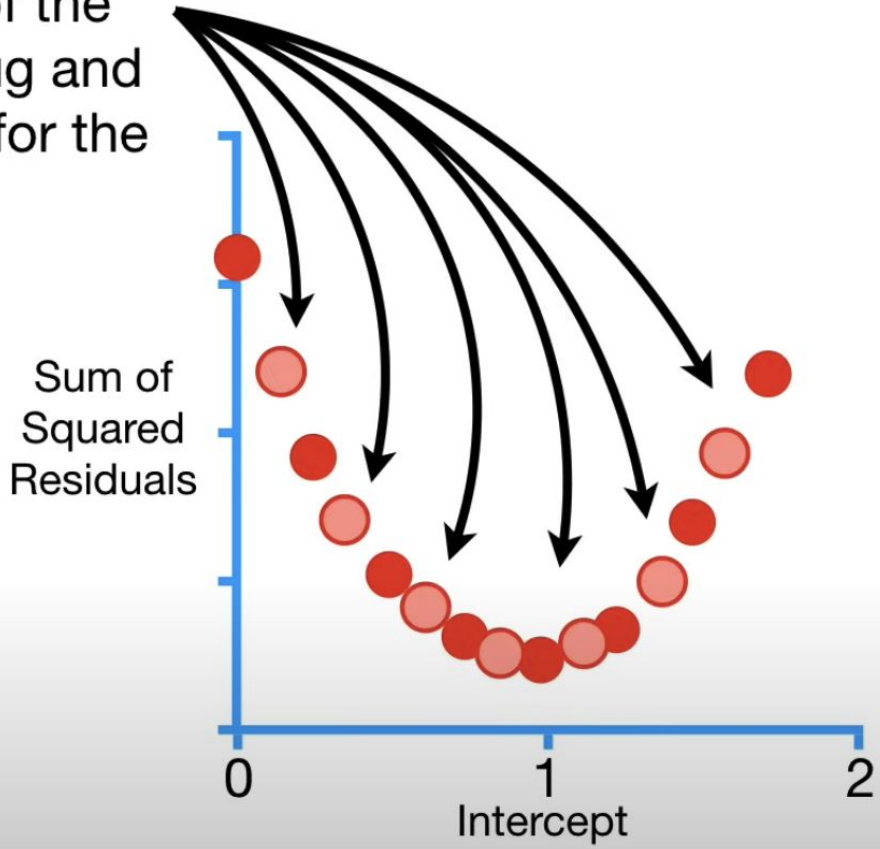



Of the points that we calculated for the graph, this one has the lowest Sum of Squared Residuals…
Sum of Squared Residuals
0
1
2
Intercept

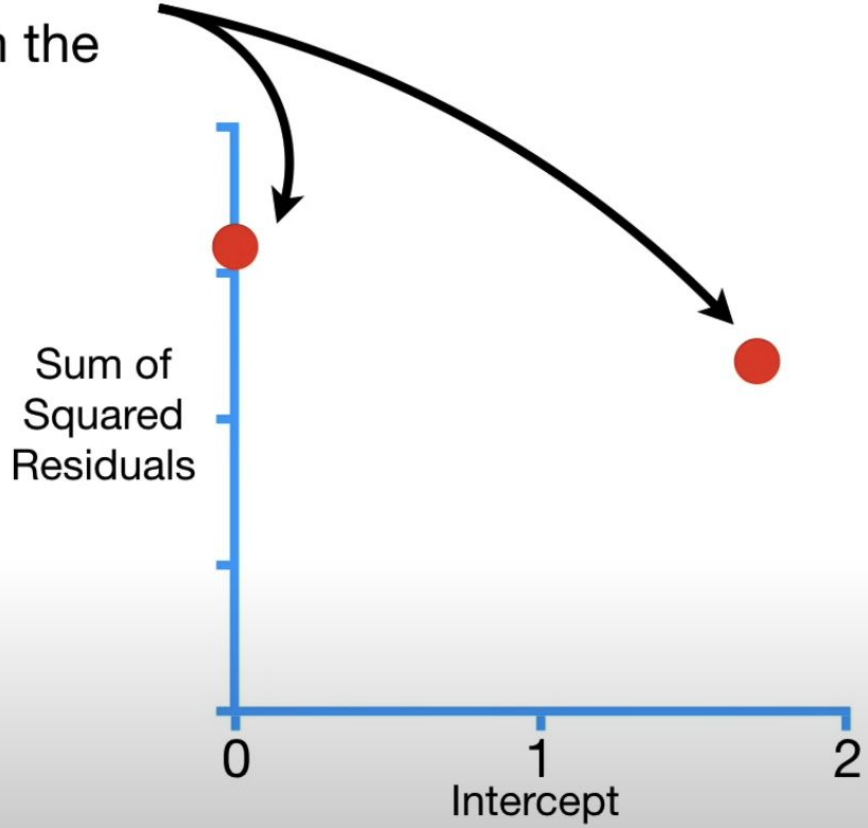
What if the best value for the **Intercept** is somewhere between these values?



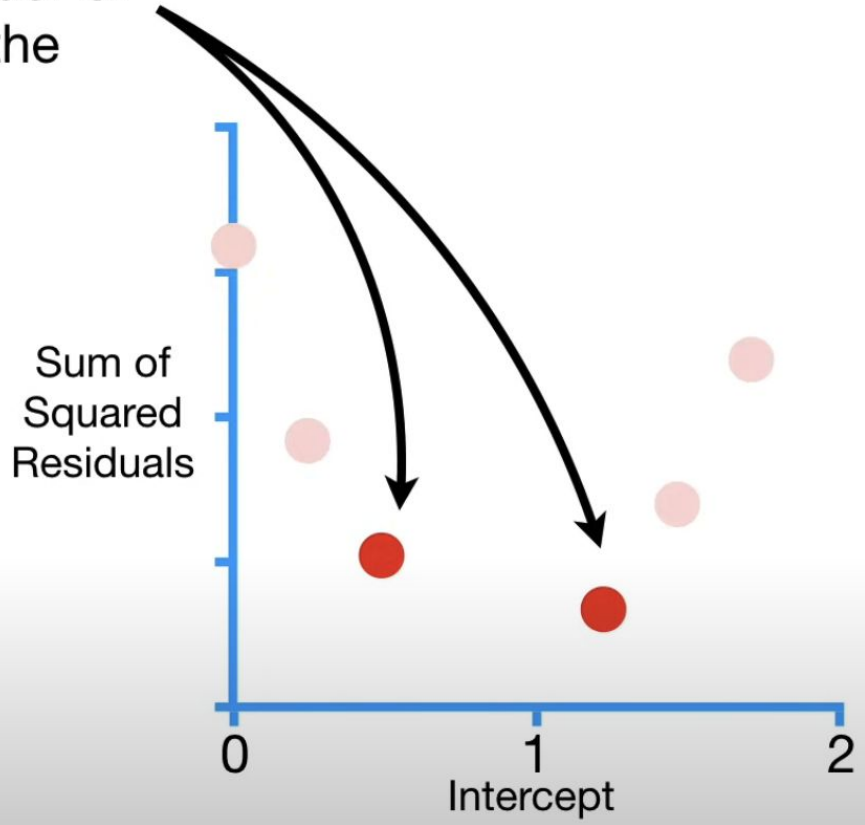
A slow and painful method for finding the minimal Sum of the Squared Residuals is to plug and chug a bunch more values for the **Intercept**.



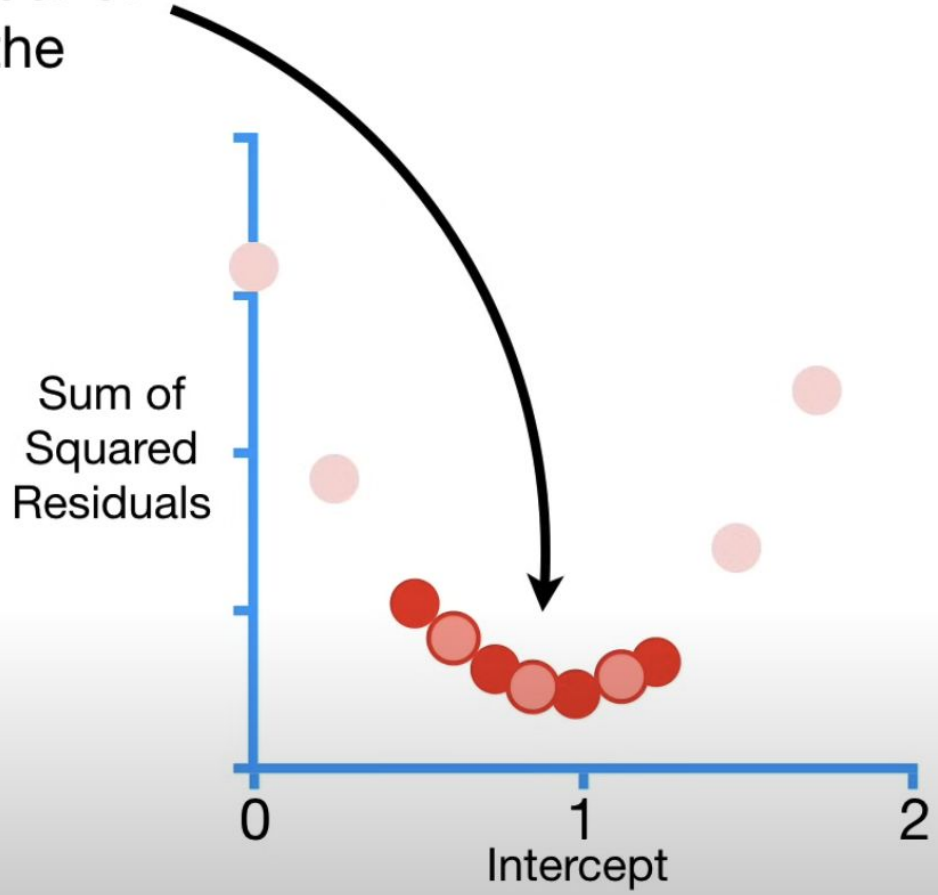
Gradient Descent only does a few calculations far from the optimal solution…



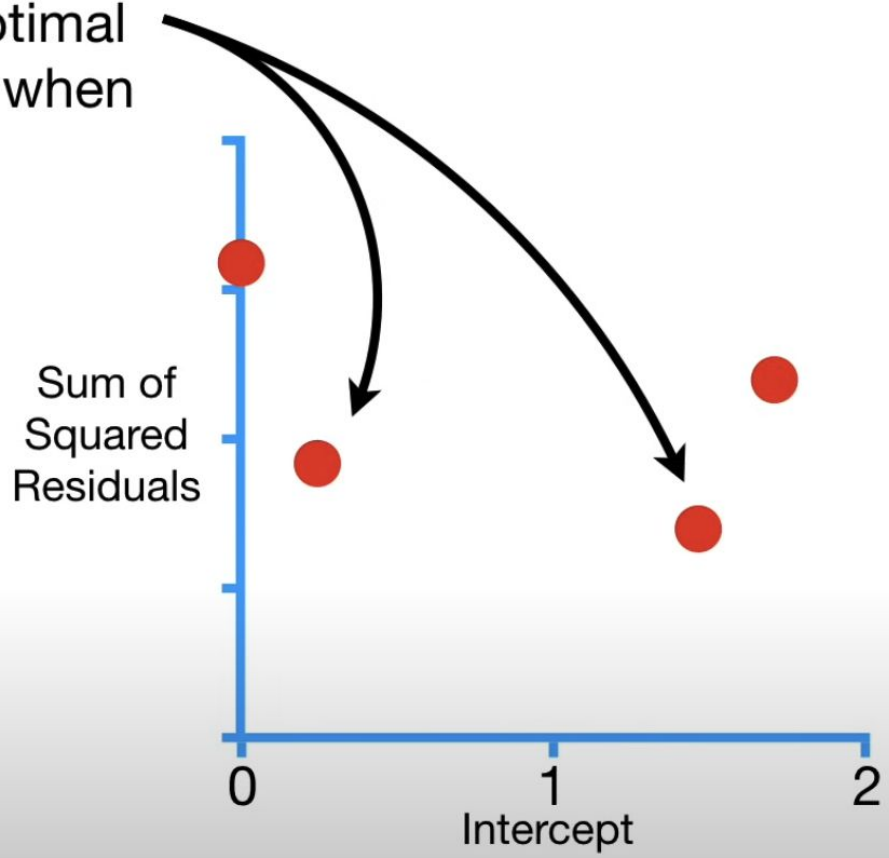
…and increases the number of calculations closer to the optimal value.

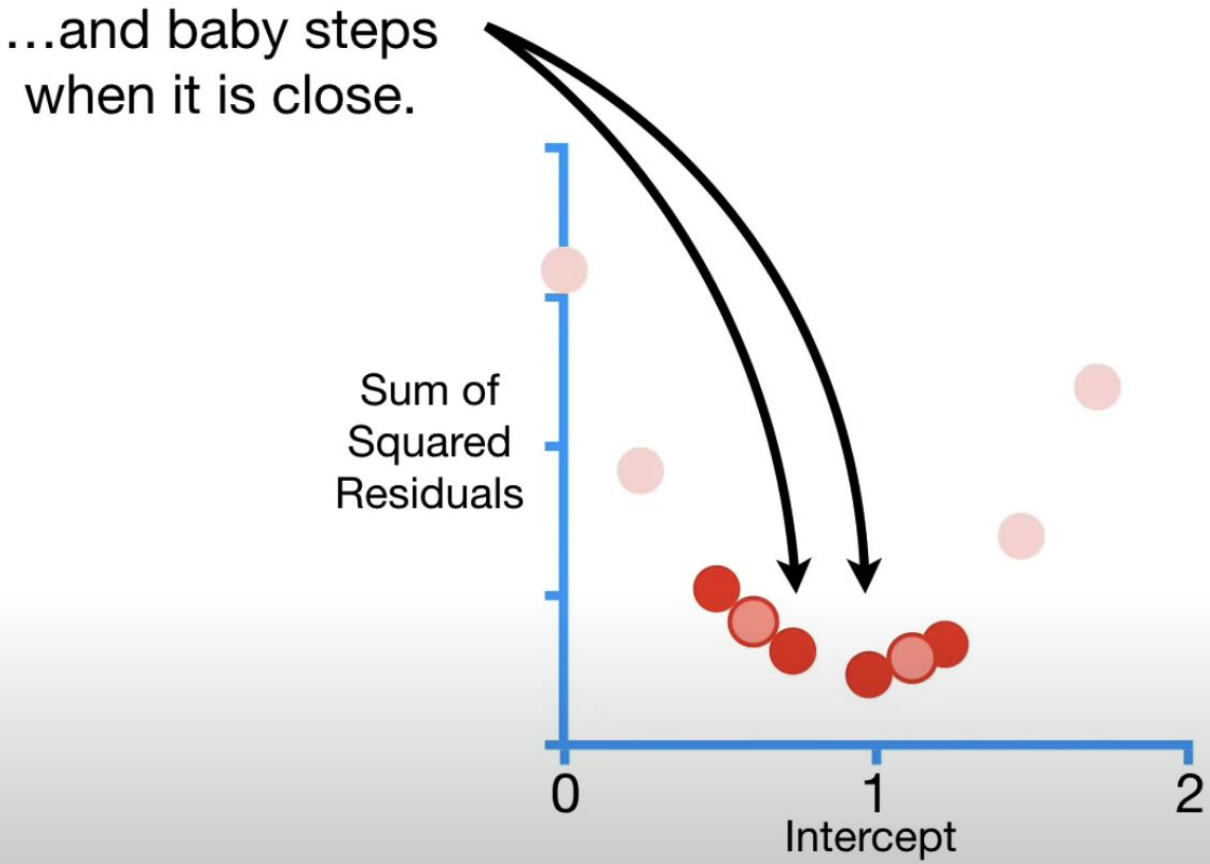


…and increases the number of calculations closer to the optimal value.

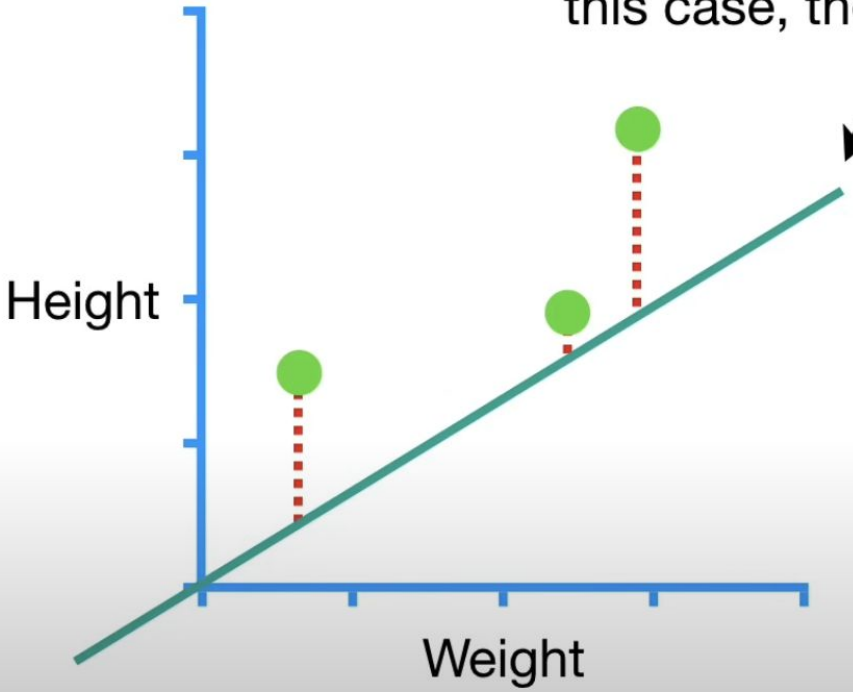


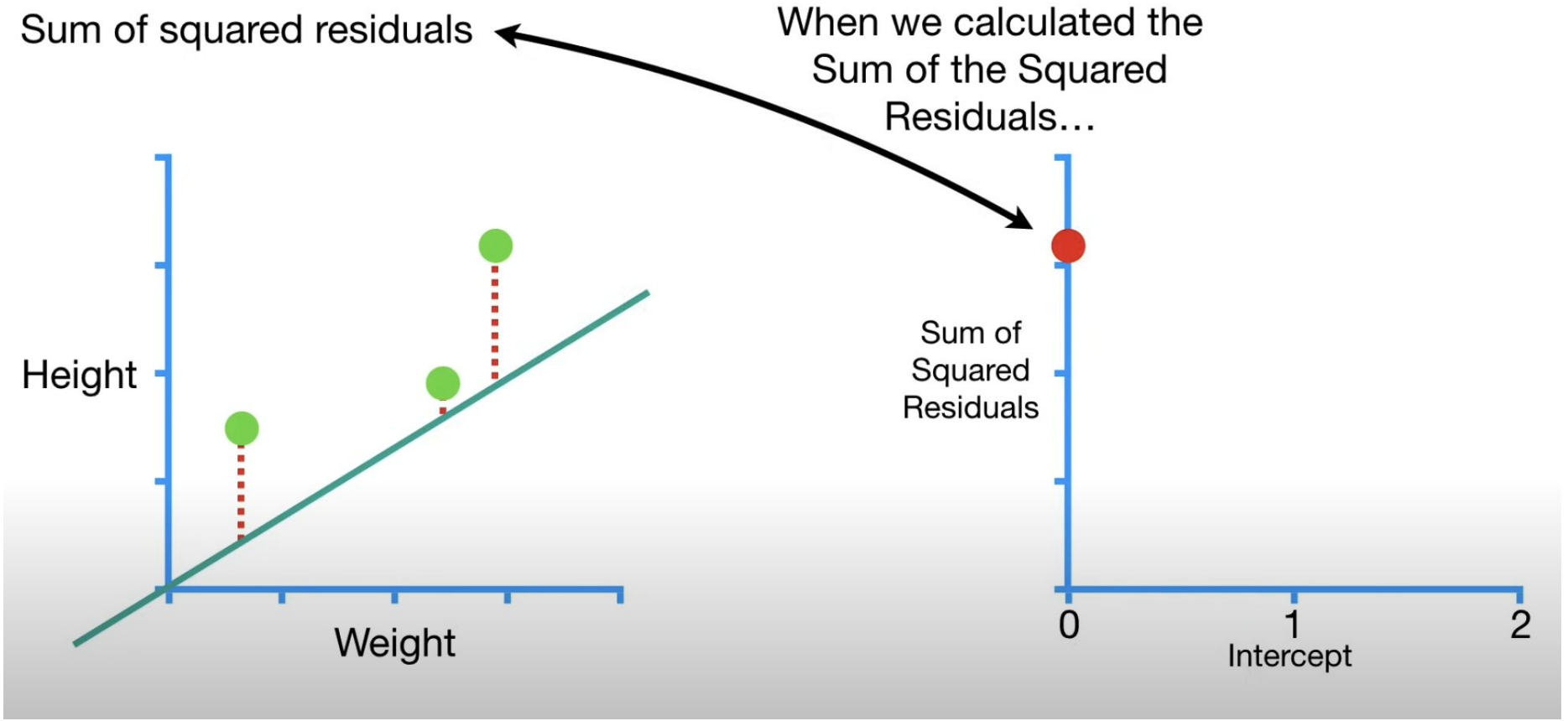
In other words, **Gradient Descent** identifies the optimal value by taking big steps when it is far away…



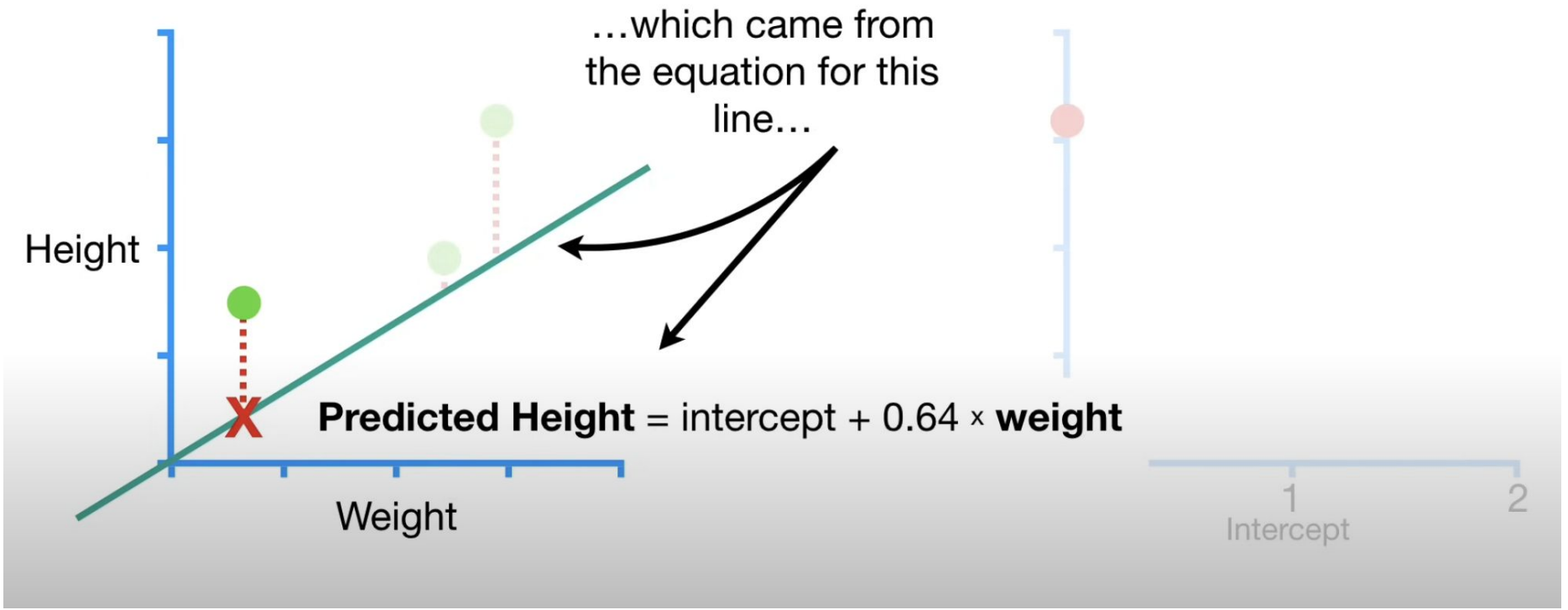

…and baby steps
when it is close.
Sum of
Squared
Residuals
0
1
2
Intercept

So let's get back to using **Gradient Descent** to find the optimal value for the **Intercept**, starting from a random value. In this case, the random value was **0**.

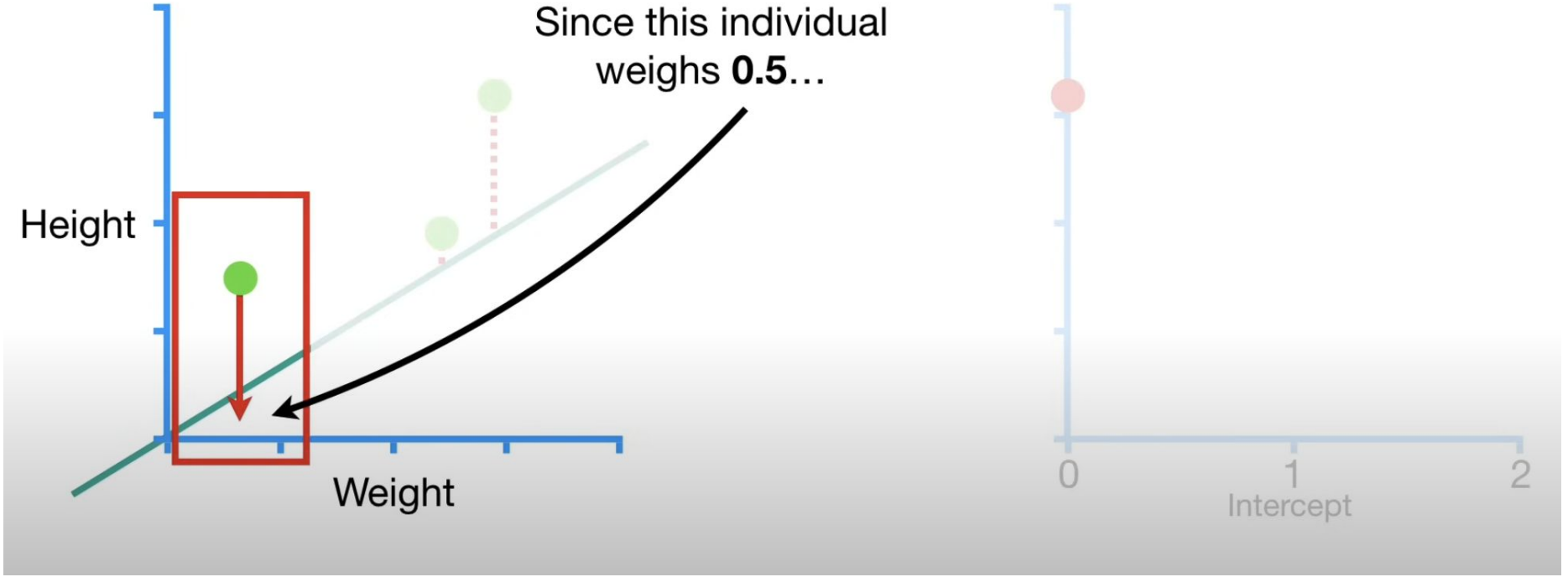



Sum of squared residuals
When we calculated the Sum of the Squared Residuals…
Height
Weight
Sum of Squared Residuals
0
1
2
Intercept

Sum of squared residuals = (**1.4** - predicted)^2

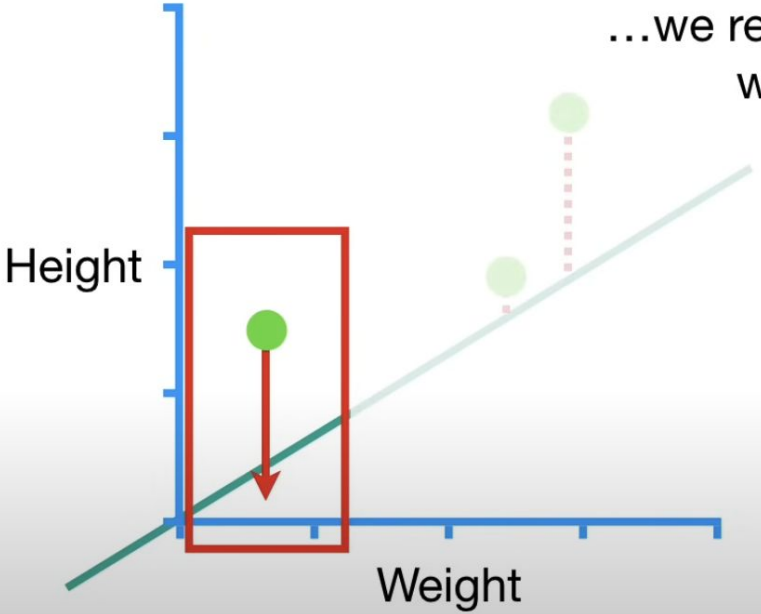


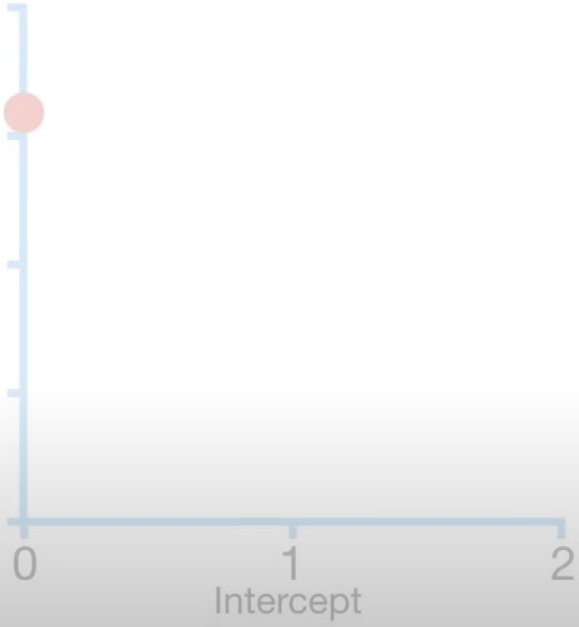
Sum of squared residuals = (**1.4** - (intercept + 0.64 × **weight**))^2



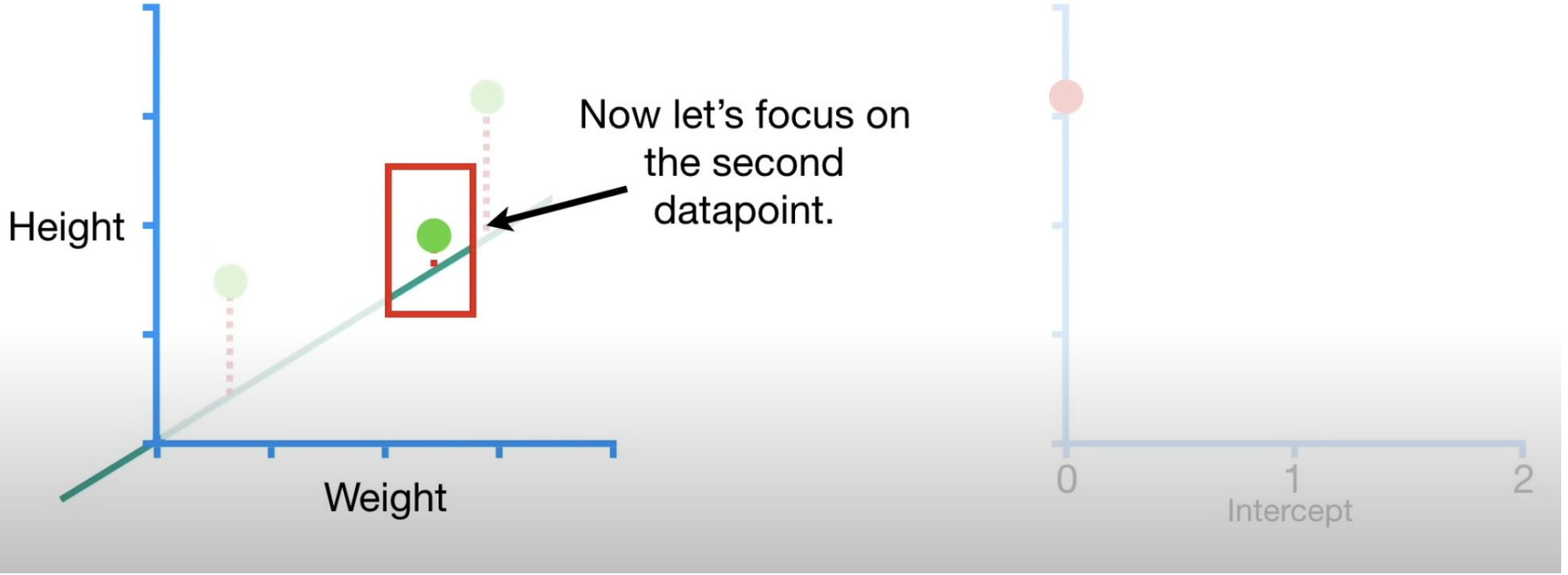
Sum of squared residuals = (**1.4** - (intercept + 0.64 x **0.5**))^2

…we replace **weight** with **0.5**.



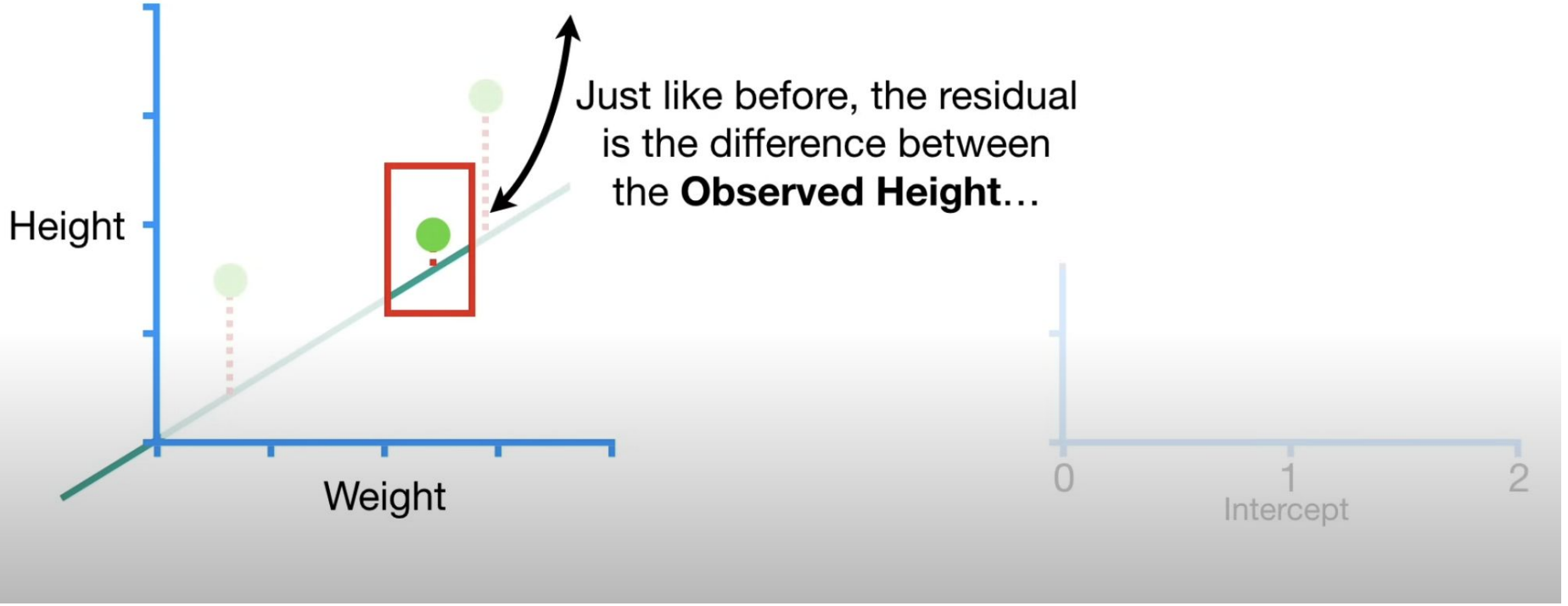


Sum of squared residuals = (**1.4** - (intercept + 0.64 x **0.5**))^2



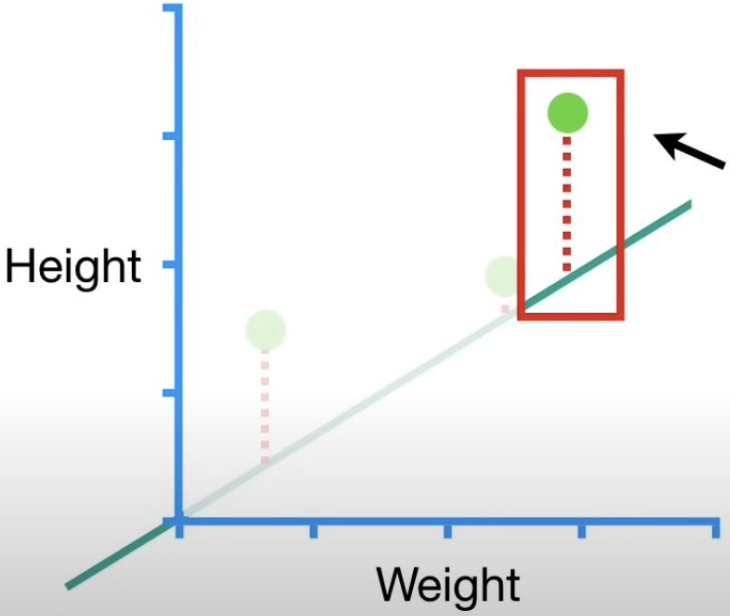
Sum of squared residuals = (**1.4** - (intercept + 0.64 x **0.5**))^2

+ (observed - predicted)^2

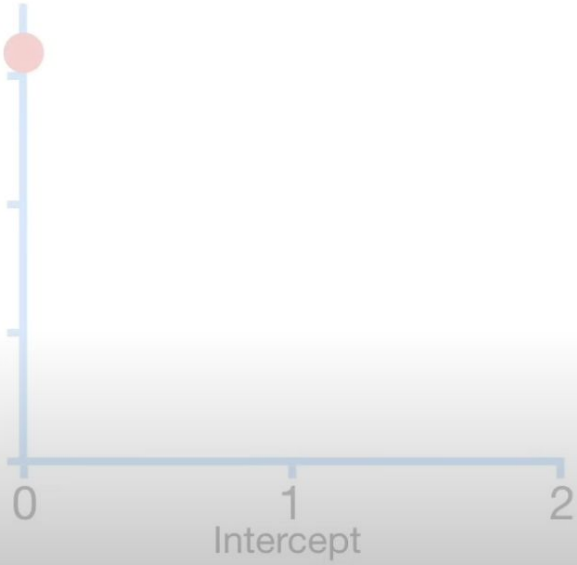


Sum of squared residuals = (**1.4** - (intercept + 0.64 x **0.5**))^2

+ (**1.9** - (intercept + 0.64 x **2.3**))^2



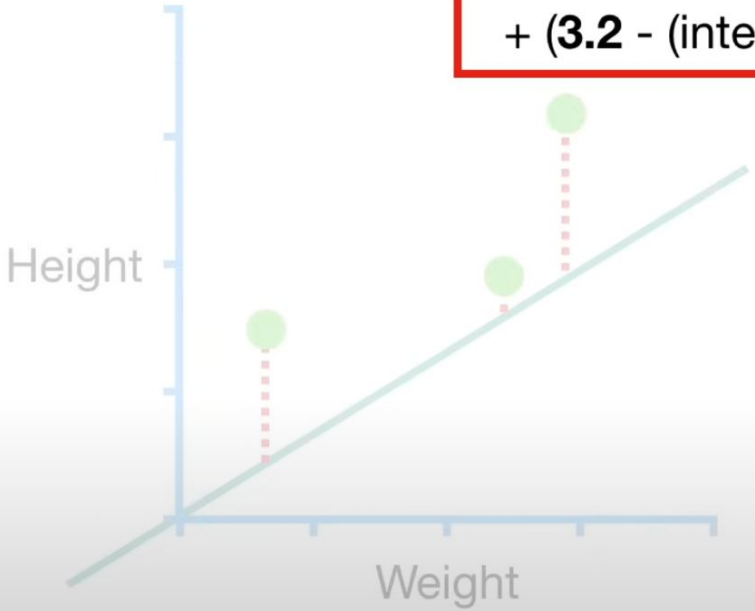
Now let's focus on the last person.



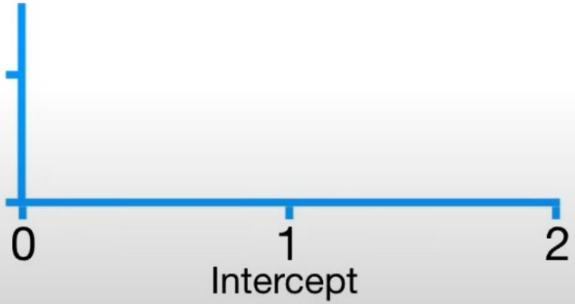
Sum of squared residuals = (**1.4** - (intercept + 0.64 × **0.5**))^2

+ (**1.9** - (intercept + 0.64 × **2.3**))^2

+ (**3.2** - (intercept + 0.64 × **2.9**))^2



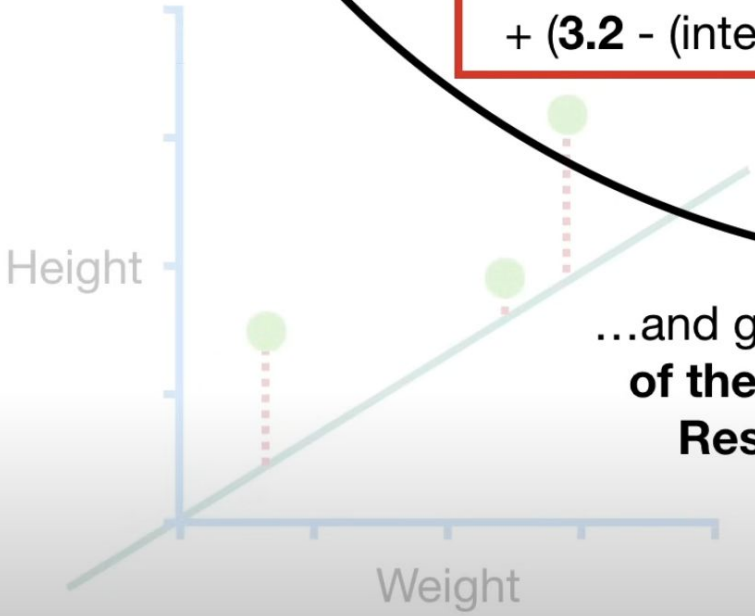
Now we can easily plug in any value for the **intercept**…



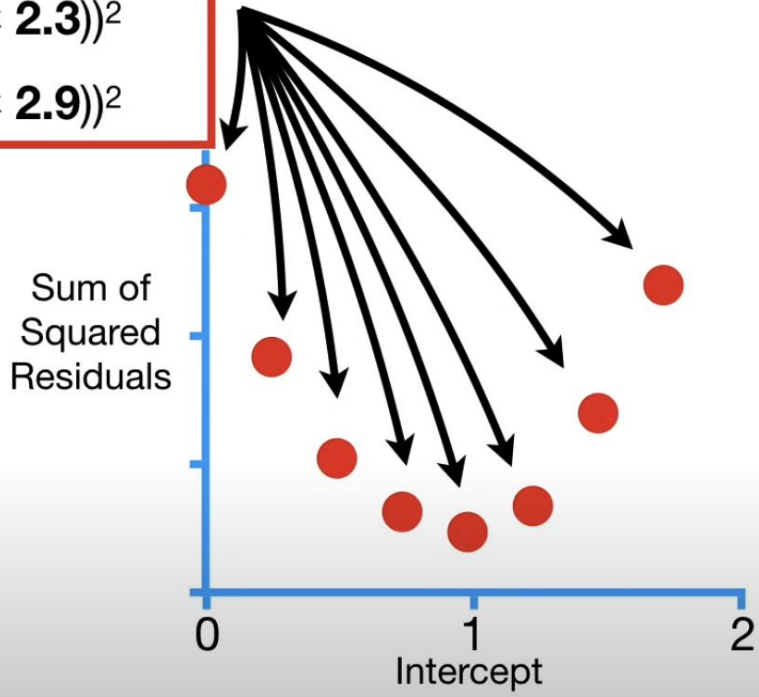
Sum of squared residuals = (**1.4** - (intercept + 0.64 × **0.5**))^2

+ (**1.9** - (intercept + 0.64 × **2.3**))^2

+ (**3.2** - (intercept + 0.64 × **2.9**))^2



…and get the **Sum of the Squared Residuals**.

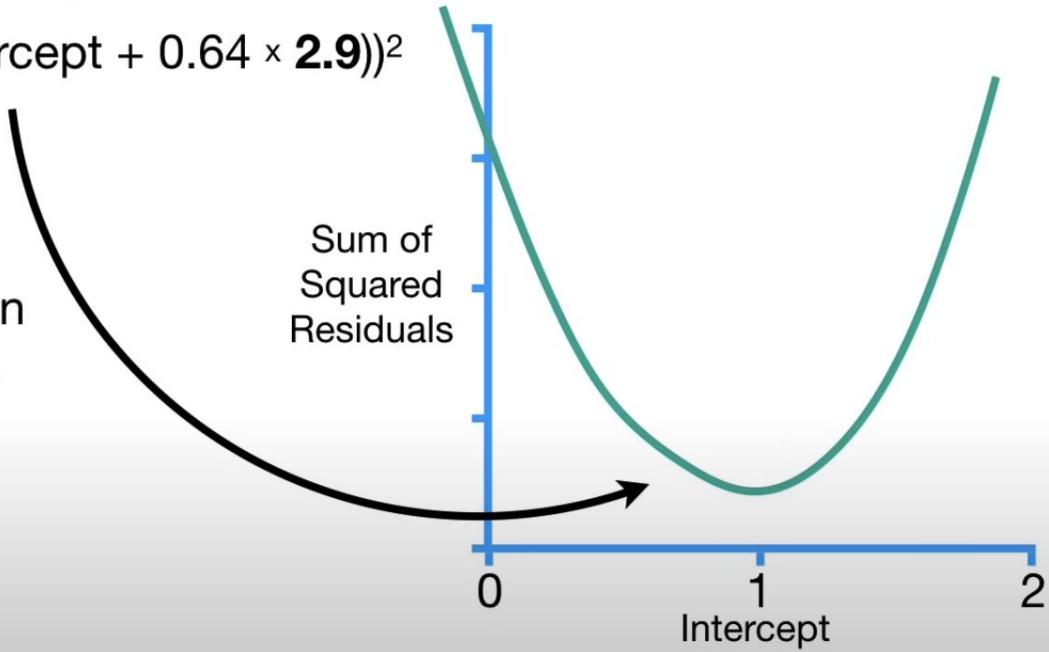


Sum of squared residuals = (**1.4** - (intercept + 0.64 × **0.5**))^2

+ (**1.9** - (intercept + 0.64 × **2.3**))^2

+ (**3.2** - (intercept + 0.64 × **2.9**))^2

Thus, we now have an equation for this curve…

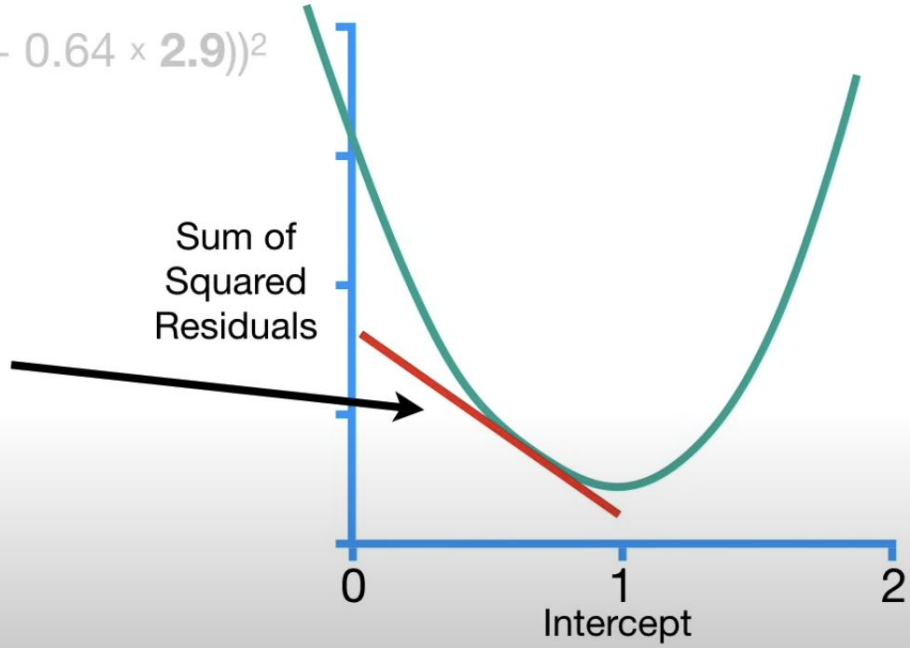


Sum of squared residuals = (**1.4** - (intercept + 0.64 x **0.5**))^2

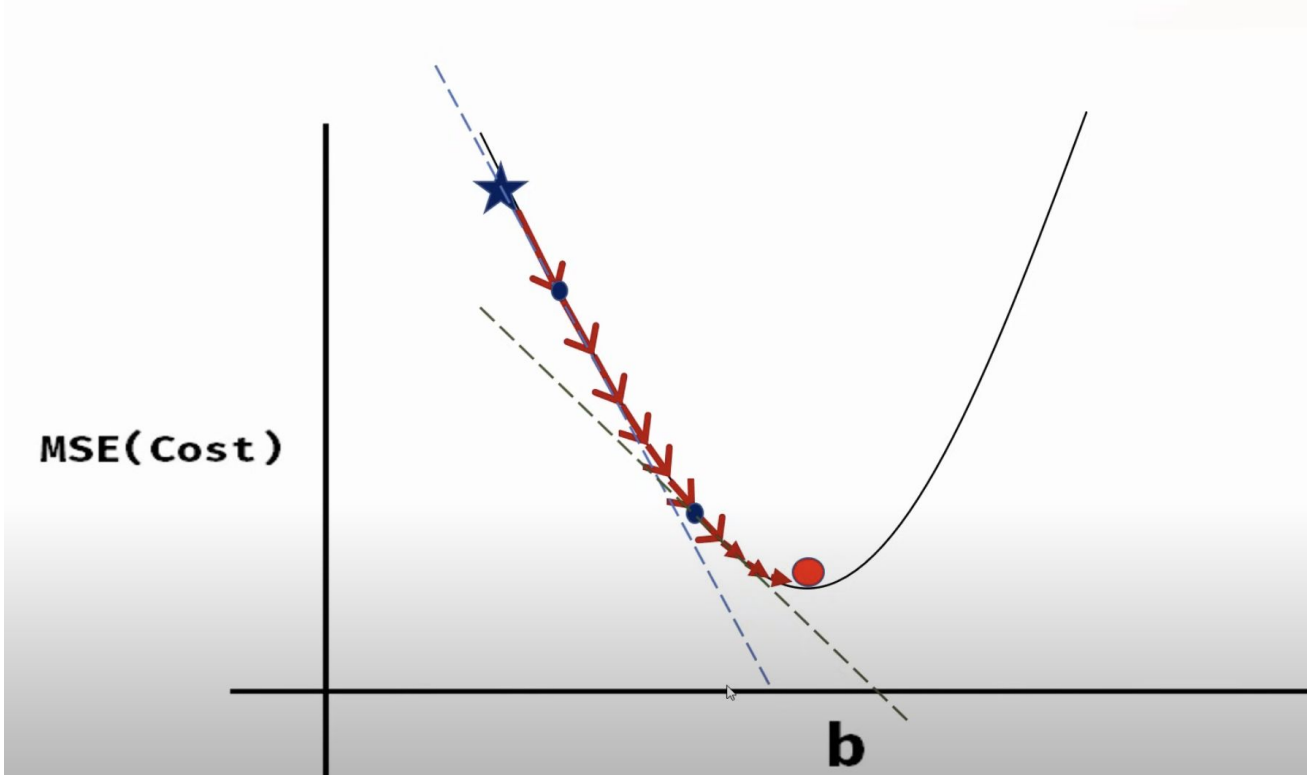
+ (**1.9** - (intercept + 0.64 x **2.3**))^2

+ (**3.2** - (intercept + 0.64 x **2.9**))^2

…and we can take the derivative of this function and determine the slope at any value for the **Intercept**.



Need to calculate



Finding the derivatives

$$ms\varepsilon = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - (mx_i + b)\right)^2$$

$$\partial/_{\partial m} = \frac{2}{\mathrm{n}}\sum_{i=1}^{n} -x_i\left(y_i - (mx_i + b)\right)$$

$$\partial/_{\partial b} = \frac{2}{\mathrm{n}}\sum_{i=1}^{n} -\left(y_i - (mx_i + b)\right)$$

Calculation of slope and intercept

Using partial derivative

$$m = m - \text{learning rate} * \partial/\partial m$$

$$b = b - \text{learning rate} * \partial/\partial b$$

For the given dataset, try to fit the linear regression.

x	y
1	2
2	3
3	5
4	7

Try to fit in the linear regression:

$y = a + bx$

Initialize the guesses: m=0, b=0

Initialize the learning rate (alpha) = 0.01

Cost Function:

MSE = 1/n* (∑(Actual - Predicted)^2)

Compute the gradients using the formula:
$\partial m/\partial J = -2/n * \sum (xi\ (yi - (m*xi+b))$

$\partial J/\partial b = -2/n * \sum (yi - (m*xi+b))$

Update the gradients:
$m := m - \eta * \partial m / \partial J$

$b := b - \eta * \partial J / \partial b$

Iteration 1:
For each data point, For each data point, compute

$$y' = mx + b = 0x + 0 = 0$$

Calculate Errors:

- For $x=1, y=2: e=2-0=2$
- For $x=2, y=3: e=3-0=3$
- For $x=3, y=5: e=5-0=5$
- For $x=4, y=7: e=7-0=7$

Compute Gradients:

Average error for m:

$$\partial J/\partial m = -2/4(1*2+2*3+3*5+4*7) = -2/4(2+6+15+28) = -2/4(51) = -25.5$$

Average error for b:

$$\partial J/\partial b = -2/4(2+3+5+7) = -2/4(17) = -8.5$$

Update Parameters

- Update m:

$$m := m - (0.01)(-25.5) = 0 + 0.255 = 0.255$$

- Update b:

$$b := b - (0.01)(-8.5) = 0 + 0.085 = 0.085$$

Repeat this iteration for about say 100 times, they will converge to a point.

Suppose,
$m \approx 1.75$
$b \approx 0.25$

The resulting linear regression model would be:

$$y \approx 1.75x + 0.25$$

$$y \approx 1.75x + 0.25$$

Multivariate Linear Regression

Multivariate Linear Regression is an extension of simple linear regression that models the relationship between two or more independent variables (predictors) and a single dependent variable (response) using a linear equation. It is used when we want to predict a continuous output based on multiple input features.

Mathematical Model

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$$

- Y: Dependent variable (target/output)
- $X_1, X_2, \ldots, X_n$: Independent variables (features)
- β_0: Intercept
- $\beta_1, \beta_2, \ldots, \beta_n$: Coefficients for each independent variable
- ε: Error term (residual)

Ordinary Least Squares (OLS) Estimation

OLS estimates β by minimizing the residual sum of squares (RSS):

$$\text{RSS} = (\mathbf{y} - \mathbf{X}\beta)^\top (\mathbf{y} - \mathbf{X}\beta)$$

To minimize, take the derivative with respect to β and set to zero:

$$\frac{\partial \text{RSS}}{\partial \beta} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\beta) = 0$$

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Numerical Example

Suppose we have the following data with 3 observations and 2 predictors:

Observation	x_1	x_2	y
1	1	2	6
2	2	1	8
3	3	3	12

Step 1: Define Matrices

Design matrix X:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \end{bmatrix}$$

Response vector y:

$$\mathbf{y} = \begin{bmatrix} 6 \\ 8 \\ 12 \end{bmatrix}$$

Step 2: Compute $\mathbf{X}^\top\mathbf{X}$

First, compute $\mathbf{X}^\top$:

$$\mathbf{X}^\top = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & 3 \end{bmatrix}$$

Now compute:

$$\mathbf{X}^\top\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 6 \\ 6 & 14 & 13 \\ 6 & 13 & 14 \end{bmatrix}$$

Step 3: Compute $\mathbf{X}^\top\mathbf{y}$

$$\mathbf{X}^\top\mathbf{y} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & 3 \end{bmatrix} \begin{bmatrix} 6 \\ 8 \\ 12 \end{bmatrix} = \begin{bmatrix} 6+8+12 \\ 6+16+36 \\ 12+8+36 \end{bmatrix} = \begin{bmatrix} 26 \\ 58 \\ 56 \end{bmatrix}$$

Step 4: Compute $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Given:

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \begin{bmatrix} 2.333 & -1.000 & -1.000 \\ -1.000 & 0.833 & 0.000 \\ -1.000 & 0.000 & 0.833 \end{bmatrix}$$

Then:

$$\hat{\beta} = \begin{bmatrix} 2.333 & -1.000 & -1.000 \\ -1.000 & 0.833 & 0.000 \\ -1.000 & 0.000 & 0.833 \end{bmatrix} \begin{bmatrix} 26 \\ 58 \\ 56 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Final Regression Equation

$$\hat{y} = 2 + 1 \cdot x_1 + 2 \cdot x_2$$

Step 5: Predictions and Residuals

- Observation 1: $\hat{y} = 2 + 1 \cdot 1 + 2 \cdot 2 = 2 + 1 + 4 = 7$, Actual $y = 6$ Residual $= -1$
- Observation 2: $\hat{y} = 2 + 1 \cdot 2 + 2 \cdot 1 = 2 + 2 + 2 = 6$, Actual $y = 8$ Residual $= +2$
- Observation 3: $\hat{y} = 2 + 1 \cdot 3 + 2 \cdot 3 = 2 + 3 + 6 = 11$, Actual $y = 12$ Residual $= +1$

1. R-squared (R^2)

Definition:

R-squared measures the proportion of the variance in the dependent variable y that is explained by the model.

$$R^2 = 1 - \frac{\text{SSR}}{\text{SST}} = \frac{\text{SSE}}{\text{SST}}$$

Where:

$$\text{SSR} = \sum (y_i - \hat{y}_i)^2 \quad \text{(Sum of Squared Residuals)}$$
$$\text{SST} = \sum (y_i - \bar{y})^2 \quad \text{(Total Sum of Squares)}$$
$$\text{SSE} = \sum (\hat{y}_i - \bar{y})^2 \quad \text{(Explained Sum of Squares)}$$

- $R^2 = 1$: perfect fit
- $R^2 = 0$: model explains no variance (same as the mean)
- $R^2 < 0$: model is worse than using the mean (can happen if no intercept is used)

2. Adjusted R-squared

Definition:
Adjusted R^2 penalizes the R-squared value based on the number of predictors to discourage overfitting.

$$\bar{R}^2 = 1 - \left(\frac{(1 - R^2)(n-1)}{n-p} \right)$$

Where:

- n = number of observations
- p = number of parameters (including intercept)

From the model: $\hat{y} = 2 + 1x_1 + 2x_2$ considering we got different predicted $\hat{y}$ as 7 6 and 11

Observation	y	$\hat{y}$	Residual $(y - \hat{y})$
1	6	7	-1
2	8	6	+2
3	12	11	+1

Step 1: SSR (Sum of Squared Residuals)

$$\text{SSR} = (-1)^2 + 2^2 + 1^2 = 1 + 4 + 1 = 6$$

Step 2: SST (Total Sum of Squares)

$$\bar{y} = \frac{6+8+12}{3} = \frac{26}{3} \approx 8.67$$

$$\text{SST} = (6-8.67)^2 + (8-8.67)^2 + (12-8.67)^2 = (-2.67)^2 + (-0.67)^2 + (3.33)^2 = \approx 18.65$$

Step 3: Compute R^2

$$R^2 = 1 - \frac{6}{18.65} \approx 0.678$$

Multivariate Linear Regression Example

We want to predict a student's final exam score (Y) based on:

- Hours studied (X_1)
- Number of practice exams taken (X_2)

Sample Data

Student	Hours (X_1)	Practice Exams (X_2)	Final Score (Y)
1	2	1	60
2	3	2	75
3	4	2	85
4	5	3	95
5	6	4	100

Step 1: Matrix Setup

Design matrix $\mathbf{X}$ and response vector $\mathbf{Y}$:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 2 \\ 1 & 4 & 2 \\ 1 & 5 & 3 \\ 1 & 6 & 4 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 60 \\ 75 \\ 85 \\ 95 \\ 100 \end{bmatrix}$$

Step 2: Compute $\mathbf{X}^\top\mathbf{X}$

$$\mathbf{X}^\top\mathbf{X} = \begin{bmatrix} 5 & 20 & 12 \\ 20 & 90 & 53 \\ 12 & 53 & 34 \end{bmatrix}$$

Step 3: Compute $\mathbf{X}^\top\mathbf{Y}$

$$\mathbf{X}^\top\mathbf{Y} = \begin{bmatrix} 415 \\ 1690 \\ 1025 \end{bmatrix}$$

Step 4: Calculate Regression Coefficients

Using $\hat{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{Y}$:

$$\hat{\beta} = \begin{bmatrix} 35 \\ 10 \\ 5 \end{bmatrix}$$

Final regression equation:

$$\hat{Y} = 35 + 10X_1 + 5X_2$$

Model Evaluation

- Mean of Y: $\bar{Y} = 83$
- Total Sum of Squares (SST): 1030
- Residual Sum of Squares (SSE): 250
- Regression Sum of Squares (SSR): 780

Step 6: Calculate R^2 and Adjusted R^2

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{250}{1030} \approx 0.7573 \quad (75.73\%)$$

$$R^2_{adj} = 1 - \left(\frac{n-1}{n-p-1}\right)(1-R^2) = 1 - \left(\frac{4}{2}\right)(0.2427) \approx 0.5146 \quad (51.46\%)$$

Step 7: Predictions and Residuals

Student	Actual Y	Predicted $\hat{Y}$	Residual
1	60	35+10(2)+5(1)=60	0
2	75	35+10(3)+5(2)=75	0
3	85	35+10(4)+5(2)=85	0
4	95	35+10(5)+5(3)=100	-5
5	100	35+10(6)+5(4)=115	-15

Interpretation

- Both hours studied and practice exams have positive effects on final scores
- The model explains 75.73% of variance in exam scores
- Adjusted R^2 accounts for the small sample size, showing 51.46% explanatory power
- The model fits perfectly for the first 3 observations but has larger residuals for the last 2

Multicollinearity and Variance Inflation Factors (VIF)

Multicollinearity occurs when two or more predictor variables in a regression model are highly linearly related.

This can cause the coefficient estimates to be unstable and their variances to be inflated, making it difficult to assess the importance of individual predictors.

Problems Caused due to this:

- Coefficients become sensitive to small changes in the model or data.
- Interpretation of the coefficients becomes unreliable.
- Increases standard errors of the coefficients.

Variance Inflation Factor (VIF)

The Variance Inflation Factor for the j-th predictor is defined as:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

Where R_j^2 is the R-squared from regressing predictor j on all the other predictors.

Interpretation of VIF

VIF Value	Interpretation
1	No multicollinearity
1 to 5	Moderate correlation, acceptable
5 to 10	High correlation, potential problem
greater than 10	Serious multicollinearity issue

Multicollinearity and Variance Inflation Factors (VIF)

Multicollinearity occurs when predictor variables in a regression model are highly correlated. This can lead to unreliable estimates of regression coefficients. A common diagnostic tool for detecting multicollinearity is the **Variance Inflation Factor (VIF)**.

Interpretation

- Variables `x1` and `x2` have very high VIF values, indicating strong multicollinearity.
- Variable `x3` has a low VIF, showing no significant correlation with other predictors.

Guidelines

- **VIF greater than 5:** Potential multicollinearity.
- **VIF greater than 10:** Serious multicollinearity problem.

Non-Parametric Regression: Nadaraya-Watson Kernel Regression

Kernel regression is a non-parametric approach used to estimate the relationship between variables.

The Nadaraya-Watson kernel regression is a popular method that provides a smooth estimate of the conditional expectation of the response variable.

Derivation of the estimator

The Nadaraya-Watson estimator for non-parametric regression can be derived as follows:

Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, the goal is to estimate the conditional expectation $\mathbb{E}[Y|X = x]$. The Nadaraya-Watson estimator for $\mathbb{E}[Y|X = x]$ is given by:

$$\hat{m}(x) = \frac{\sum_{i=1}^n K_h(x - x_i) y_i}{\sum_{i=1}^n K_h(x - x_i)}$$

where: - $\hat{m}(x)$ is the estimated value of $\mathbb{E}[Y|X = x]$, - $K_h(\cdot)$ is the kernel function with bandwidth h, - x_i and y_i are the data points, and - h is the bandwidth parameter that controls the smoothness of the estimate.

The kernel function $K_h(\cdot)$ is usually a symmetric function such as the Gaussian kernel:

$$K_h(x - x_i) = \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

where $K(u)$ is a kernel function, such as the Gaussian kernel:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}u^2\right)$$

Thus, the Nadaraya-Watson estimator can be written as:

$$\hat{m}(x) = \frac{\sum_{i=1}^{n} \frac{1}{h} K\left(\frac{x - x_i}{h}\right) y_i}{\sum_{i=1}^{n} \frac{1}{h} K\left(\frac{x - x_i}{h}\right)}$$

This equation is the basic formulation of the Nadaraya-Watson kernel regression estimator.

Rules for Setting the Appropriate Bandwidth Size

The bandwidth parameter h plays a crucial role in determining the smoothness of the estimated regression function.

A smaller bandwidth results in a less smooth estimate that is more sensitive to the data points, while a larger bandwidth produces a smoother estimate.

1. Rule of Thumb: One common method for selecting the bandwidth is the rule of thumb, which is based on minimizing the Mean Squared Error (MSE). A typical heuristic for bandwidth selection is:

$$h_{\text{opt}} \approx \left(\frac{4\hat{\sigma}^5}{3n} \right)^{1/5}$$

where: - $\hat{\sigma}^2$ is the sample variance of the response variable, - n is the number of data points.

This rule assumes that the data is roughly normally distributed and that the kernel is Gaussian.

2. Cross-validation: Cross-validation is another commonly used method to select the optimal bandwidth. In this method, the data is split into training and test sets, and the bandwidth that minimizes the cross-validation error is chosen. The cross-validation error is given by:

$$CV(h) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{m}_{-i}(x_i))^2$$

where $\hat{m}_{-i}(x_i)$ is the Nadaraya-Watson estimate computed without using the i-th data point.

Nadaraya-Watson kernel estimator

The Nadaraya-Watson kernel regression is a powerful non-parametric method for

estimating the conditional expectation of the response variable.

The choice of bandwidth h is critical in determining the smoothness of the estimate and can be selected using rules of thumb, cross-validation, or other methods depending on the nature of the data.

Given a new dataset to demonstrate non-parametric regression:

- $X = [1, 2, 3, 4, 5]$
- $Y = [2, 4, 1, 5, 3]$

Estimate $\hat{y}$ at $x = 2.5$ using the Nadaraya-Watson kernel regression with Gaussian kernel:

$$\hat{y}(x) = \frac{\sum_{i=1}^{n} K_h(x - X_i) Y_i}{\sum_{i=1}^{n} K_h(x - X_i)}$$

where $K_h(u) = \exp\left(-\frac{u^2}{2h^2}\right)$ and bandwidth $h = 1$.

Numerical Calculation

Step 1: Compute Distances

Compute distances from query point $x_q = 2.5$ to all X_i:

$$d_1 = |2.5 - 1| = 1.5$$
$$d_2 = |2.5 - 2| = 0.5$$
$$d_3 = |2.5 - 3| = 0.5$$
$$d_4 = |2.5 - 4| = 1.5$$
$$d_5 = |2.5 - 5| = 2.5$$

Step 2: Compute Kernel Weights

Gaussian kernel: $K_h(u) = \exp\left(-\frac{u^2}{2h^2}\right)$ with $h = 1$:

$$w_1 = \exp\left(-\frac{(1.5)^2}{2 \times 1^2}\right) = \exp\left(-\frac{2.25}{2}\right) = e^{-1.125} \approx 0.3247$$

$$w_2 = \exp\left(-\frac{(0.5)^2}{2}\right) = e^{-0.125} \approx 0.8825$$

$$w_3 = \exp\left(-\frac{(0.5)^2}{2}\right) = e^{-0.125} \approx 0.8825$$

$$w_4 = \exp\left(-\frac{(1.5)^2}{2}\right) = e^{-1.125} \approx 0.3247$$

$$w_5 = \exp\left(-\frac{(2.5)^2}{2}\right) = \exp\left(-\frac{6.25}{2}\right) = e^{-3.125} \approx 0.0440$$

Step 3: Compute Weighted Sums

Numerator (weighted Y sum):

$$\sum w_i Y_i = (0.3247 \times 2) + (0.8825 \times 4) + (0.8825 \times 1) + (0.3247 \times 5) + (0.0440 \times 3)$$

$$= 0.6494 + 3.5300 + 0.8825 + 1.6235 + 0.1320 = 6.8174$$

Denominator (sum of weights):

$$\sum w_i = 0.3247 + 0.8825 + 0.8825 + 0.3247 + 0.0440 = 2.4584$$

Step 4: Compute Estimate

$$\hat{y}(2.5) = \frac{\sum w_i Y_i}{\sum w_i} = \frac{6.8174}{2.4584} \approx 2.772$$

Principal Component Analysis

Principal Component Analysis (PCA) is a fundamental unsupervised dimensionality reduction technique widely used in statistics, machine learning, and data visualization.

It transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible.

Idea regarding PCA

PCA identifies new axes (principal components) in the data that maximize variance.

These axes are linear combinations of the original features and are orthogonal (uncorrelated) to each other.

- First Principal Component (PC1): Direction of maximum variance
- Second Principal Component (PC2): Orthogonal to PC1, captures the next highest variance

Why PCA?

- Visualization: Reduce data to 2D/3D for plotting
- Noise Reduction: Remove low-variance (noisy) dimensions
- Efficiency: Speeds up machine learning models
- Multicollinearity Removal: Produces uncorrelated features

Principal Component Analysis (PCA) Steps

1. **Standardize the data**:

$$\mathbf{X}_{\text{std}} = \frac{\mathbf{X} - \mu}{\sigma}$$

(Mean-center and scale to unit variance)

2. **Compute covariance matrix**:

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}_{\text{std}}^{\top}\mathbf{X}_{\text{std}}$$

Principal Component Analysis (PCA) Steps

3. **Perform eigendecomposition**:

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$$

(Find eigenvalues λ and eigenvectors $\mathbf{v}$)

4. **Select principal components**:

   - Sort eigenvalues in descending order
   - Choose top k eigenvectors (loading vectors) $\mathbf{W}_k$

5. **Transform the data**:

$$\mathbf{X}_{\text{PCA}} = \mathbf{X}_{\text{std}}\mathbf{W}_k$$

(Project onto new subspace)

Applications of PCA

- Data Compression: Reduce storage & computation costs
- Visualization: Plot high-dimensional data in 2D/3D
- Noise Reduction: Discard low-variance components
- Feature Extraction: Improve supervised learning

Limitations

- Interpretability Loss: PCs are linear combinations
- Scaling Sensitivity: Must standardize data first
- Linearity Assumption: Nonlinear methods may be better
- Information Loss: Dropping too many PCs loses variance

PCA Algorithm (For calculation)

The steps involved in PCA Algorithm are as follows:

Step-01: Get data.

Step-02: Compute the mean vector (μ).

Step-03: Subtract mean from the given data.

Step-04: Calculate the covariance matrix.

Step-05: Calculate the eigen vectors and eigen values of the covariance matrix.

Step-06: Choosing components and forming a feature vector.

Step-07: Deriving the new data set.

Consider the two dimensional patterns (2, 1), (3, 5), (4, 3), (5, 6), (6, 7), (7,8). Use PCA for 1-D transformation

The given feature vectors are-

$$x_1 = (2,1)$$
$$x_2 = (3,5)$$
$$x_3 = (4,3)$$
$$x_4 = (5,6)$$
$$x_5 = (6,7)$$
$$x_6 = (7,8)$$

$$\begin{bmatrix} 2 \\ 1 \end{bmatrix} \begin{bmatrix} 3 \\ 5 \end{bmatrix} \begin{bmatrix} 4 \\ 3 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} \begin{bmatrix} 6 \\ 7 \end{bmatrix} \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Step-02:

Calculate the mean vector (μ).

Mean vector $(\mu) = \left(\frac{2+3+4+5+6+7}{6}, \frac{1+5+3+6+7+8}{6}\right) = (4.5, 5)$

Thus,

$$\text{Mean vector } (\mu) = \begin{bmatrix} 4.5 \\ 5 \end{bmatrix}$$

Step-03:

Subtract mean vector (μ) from the given feature vectors.

$$x_1 - \mu = (2 - 4.5, 1 - 5) = (-2.5, -4)$$
$$x_2 - \mu = (3 - 4.5, 5 - 5) = (-1.5, 0)$$
$$x_3 - \mu = (4 - 4.5, 3 - 5) = (-0.5, -2)$$
$$x_4 - \mu = (5 - 4.5, 6 - 5) = (0.5, 1)$$
$$x_5 - \mu = (6 - 4.5, 7 - 5) = (1.5, 2)$$
$$x_6 - \mu = (7 - 4.5, 8 - 5) = (2.5, 3)$$

Feature vectors (x_i) after subtracting mean vector (μ) are:

$$\begin{bmatrix} -2.5 \\ -4 \end{bmatrix} \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} \begin{bmatrix} -0.5 \\ -2 \end{bmatrix} \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} \begin{bmatrix} 1.5 \\ 2 \end{bmatrix} \begin{bmatrix} 2.5 \\ 3 \end{bmatrix}$$

Step-04:

Calculate the covariance matrix.

Covariance matrix is given by:

$$C = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)(x_i - \mu)^T$$

$$m_1 = \begin{bmatrix} -2.5 \\ -4 \end{bmatrix} \begin{bmatrix} -2.5 & -4 \end{bmatrix} = \begin{bmatrix} 6.25 & 10 \\ 10 & 16 \end{bmatrix}$$

$$m_2 = \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} \begin{bmatrix} -1.5 & 0 \end{bmatrix} = \begin{bmatrix} 2.25 & 0 \\ 0 & 0 \end{bmatrix}$$

$$m_3 = \begin{bmatrix} -0.5 \\ -2 \end{bmatrix} \begin{bmatrix} -0.5 & -2 \end{bmatrix} = \begin{bmatrix} 0.25 & 1 \\ 1 & 4 \end{bmatrix}$$

$$m_4 = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} \begin{bmatrix} 0.5 & 1 \end{bmatrix} = \begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

$$m_5 = \begin{bmatrix} 1.5 \\ 2 \end{bmatrix} \begin{bmatrix} 1.5 & 2 \end{bmatrix} = \begin{bmatrix} 2.25 & 3 \\ 3 & 4 \end{bmatrix}$$

$$m_6 = \begin{bmatrix} 2.5 \\ 3 \end{bmatrix} \begin{bmatrix} 2.5 & 3 \end{bmatrix} = \begin{bmatrix} 6.25 & 7.5 \\ 7.5 & 9 \end{bmatrix}$$

Now,

Covariance matrix $= \frac{m_1+m_2+m_3+m_4+m_5+m_6}{6}$

On adding the above matrices and dividing by 6, we get:

$$C = \begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix}$$

Step-05:

Calculate the eigen values and eigen vectors of the covariance matrix.

λ is an eigen value for a matrix M if it is a solution of the characteristic equation $|M - \lambda I| = 0$.

So, we have:

$$\begin{vmatrix} 2.92 - \lambda & 3.67 \\ 3.67 & 5.67 - \lambda \end{vmatrix} = 0$$

From here,

$$\begin{aligned} (2.92 - \lambda)(5.67 - \lambda) - (3.67 \times 3.67) &= 0 \\ 16.56 - 2.92\lambda - 5.67\lambda + \lambda^2 - 13.47 &= 0 \\ \lambda^2 - 8.59\lambda + 3.09 &= 0 \end{aligned}$$

Solving this quadratic equation, we get $\lambda = 8.22, 0.38$

Thus, two eigen values are $\lambda_1 = 8.22$ and $\lambda_2 = 0.38$.

Clearly, the second eigen value is very small compared to the first eigen value. So, the second eigen vector can be left out.

Eigen vector corresponding to the greatest eigen value is the principal component for the given data set. So, we find the eigen vector corresponding to eigen value λ_1.

We use the following equation to find the eigen vector:

$$MX = \lambda X$$

where:

- M = Covariance Matrix
- X = Eigen vector
- λ = Eigen value

Substituting the values in the above equation, we get:

$$\begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = 8.22 \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$$

$$2.92X_1 + 3.67X_2 = 8.22X_1$$
$$3.67X_1 + 5.67X_2 = 8.22X_2$$

On simplification, we get:

$$5.3X_1 = 3.67X_2 \quad (1)$$
$$3.67X_1 = 2.55X_2 \quad (2)$$

From (1) and (2), $X_1 = 0.69X_2$

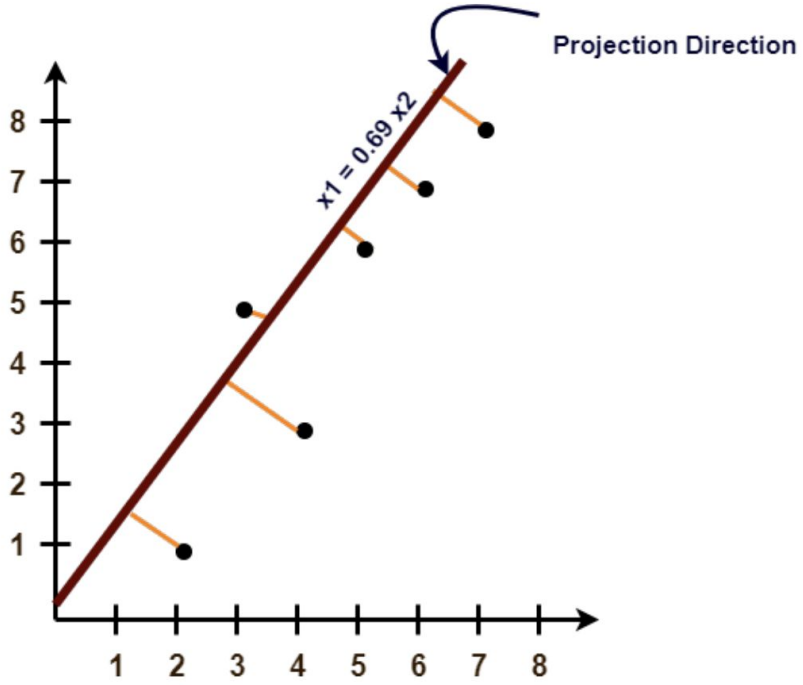
From (2), the eigen vector is:

$$X = \begin{bmatrix} 0.69 \\ 1 \end{bmatrix}$$

Thus, principal component for the given data set is:

$$\begin{bmatrix} 0.69 \\ 1 \end{bmatrix}$$

Lastly, we project the data points onto the new subspace as:



Solve this for all input vectors

The given feature vector is (2, 1).

The feature vector gets transformed to:

$$\text{Transpose of Eigen vector} \times (\text{Feature Vector} - \text{Mean Vector})$$

$$\begin{bmatrix} 0.69 & 1 \end{bmatrix} \begin{bmatrix} 2-4.5 \\ 1-5 \end{bmatrix} = \begin{bmatrix} 0.69 & 1 \end{bmatrix} \begin{bmatrix} -2.5 \\ -4 \end{bmatrix} = -5.725$$

CART (Classification and Regression Trees)

The CART (Classification and Regression Trees) algorithm is a decision tree learning method introduced by Breiman et al. (1986).

It can be used for both:

- Classification (predicting class labels), and
- Regression (predicting continuous values).

Unlike algorithms such as ID3 or C4.5, CART produces strictly binary trees—each internal node splits into exactly two children.

1. Gini Impurity (for classification)

CART uses **Gini Impurity** to measure the impurity of a dataset. It is calculated as:

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Where:

- S is the dataset at a node,
- p_i is the proportion of samples belonging to class i,
- c is the number of classes.

A lower Gini impurity indicates a purer node.

2. Mean Squared Error (for regression)

For regression tasks, CART uses **Mean Squared Error (MSE)** as the splitting criterion:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y})^2$$

CART Algorithm

1. Start with the full dataset at the root node.

2. For each feature and potential split value, compute the impurity (Gini or MSE).

3. Select the split that minimizes impurity in the resulting child nodes.

4. Split the dataset into two child nodes based on this criterion.

5. Recurse on each child node until a stopping condition is met:

- All samples belong to the same class,
- Maximum tree depth is reached,
- Minimum number of samples per node is not met,
- No further information gain is possible.

Variable Importance: Permutation Tests and Partial Dependence Plots

1. Permutation Feature Importance

To quantify the importance of each feature by measuring the change in model performance when the feature’s values are randomly shuffled, breaking the relationship between that feature and the target.

Algorithm

1. Train the model on training data.

2. Evaluate a baseline performance metric (e.g., accuracy, RMSE):

$$M_{\text{base}} = \text{metric}(y, \hat{y})$$

3. For each feature X_i:

   - Shuffle its values in the validation/test set.
   - Recalculate predictions and evaluate the performance metric M_i.
   - Compute the importance score as:

$$I_i = M_i - M_{\text{base}}$$

2. Partial Dependence Plots (PDP)

To visualize the marginal effect of one or two features on the predicted outcome by averaging out the effects of all other features.

Let $f(x)$ be the trained prediction function. The partial dependence of feature subset X_s is given by:

$$PD(X_s) = \mathbb{E}_{X_c}[f(X_s, X_c)]$$

Where:

- X_s: The feature(s) of interest.
- X_c: The complement set of all other features.
- The expectation is taken over the joint distribution of X_c.

Procedure

1. Select a feature X_s and a range of values for it.
2. For each value, substitute it in all rows of the dataset and make predictions.
3. Average the predictions to estimate the marginal effect at that value.

Comparison Table

Aspect	Permutation Importance	Partial Dependence Plot
Type	Quantitative metric	Visual interpretation
Output	Importance score	Plot of average model output
Captures Interactions	Yes	Yes (in 2D plots)
Feature Correlation Issue	Yes (underestimates importance)	Yes (can be misleading)
Model Compatibility	Any (model-agnostic)	Mostly tree-based models
Usefulness	Feature ranking	Understanding effect on prediction
Cost	Medium to High	Medium